

Hors de la Caverne

Mathématiques discrètes et informatique

Des problèmes, pas de réponses. Chaque problème ci-dessous est posé sans solution. Les références indiquent où trouver les connaissances nécessaires.

Collège

Problème 1 (Trier un jeu de cartes, et démontrer que ça marche — Collège). **DCS-51. Problème.** *Posez devant vous sept cartes de valeurs toutes différentes, faces visibles, dans un ordre quelconque. Voici une méthode pour les ranger : parcourez toutes les cartes non encore rangées, repérez la plus petite, échangez-la avec la première carte non rangée ; puis recommencez sur ce qui reste.*

- (a) *Déroulez la méthode à la main sur les valeurs 5, 3, 9, 1, 7, 2, 8, en écrivant l'état complet de la rangée après chaque échange.*
- (b) *Écrivez la phrase qui décrit ce que contient le début de la rangée et qui reste vraie après chaque échange. Démontrerez qu'elle est encore vraie après l'échange suivant, et qu'à la fin elle dit exactement « la rangée est triée ».*
- (c) *Comptez le nombre de comparaisons faites sur sept cartes, puis sur n cartes. Donnez la formule et démontrez-la.*
- (d) *Voici une seconde méthode : prenez les cartes une par une et glissez chacune à sa place parmi celles déjà tenues en main. Déroulez-la sur la même liste, puis comparez les deux méthodes sur une liste déjà triée, et sur une liste triée à l'envers.*

La seconde méthode est celle que tout joueur de cartes exécute sans y penser depuis des siècles ; la première est celle qu'un débutant réinvente au tableau. Elles portent aujourd'hui les noms de tri par insertion et de tri par sélection, et elles étaient déjà l'objet des tout premiers programmes écrits : le tri par fusion figure dans un manuscrit de John von Neumann de 1945 pour l'EDVAC, et Donald Knuth a consacré au seul problème du tri la moitié d'un volume de *The Art of Computer Programming*. Ce que ce problème demande n'est pas de trier — vous savez trier — mais de *démontrer* que la méthode trie, et la différence entre les deux est exactement le sujet de cette page. La question (b) est le premier invariant de boucle de votre vie ; DCS-02 en donne la version lycée.

Références :

- Contexte : Tri par sélection — Wikipédia (https://fr.wikipedia.org/wiki/Tri_par_s%C3%A9lection)
- Contexte : Tri par insertion — Wikipédia (https://fr.wikipedia.org/wiki/Tri_par_insertion)
- Contexte : Invariant de boucle — Wikipédia (https://fr.wikipedia.org/wiki/Invariant_de_boucle)

Problème 2 (Devine mon nombre en sept questions — Collège). **DCS-52. Problème.** *Votre adversaire choisit en secret un entier compris entre 1 et 100. Vous lui posez des questions auxquelles il ne répond que par « oui » ou par « non », et il ne ment jamais.*

- (a) *Décrivez une méthode qui trouve son nombre à coup sûr en sept questions au plus. Elle doit être assez précise pour qu'une autre personne puisse l'appliquer sans vous.*
- (b) *Démontrez qu'elle réussit toujours — pour les cent nombres possibles, et pas seulement pour ceux que vous avez essayés.*
- (c) *Démontrez qu'aucune méthode, si maligne soit-elle, ne peut garantir la victoire en six questions. (Forme suggérée : comptez ce que six réponses peuvent au plus distinguer.)*
- (d) *Jusqu'à quel nombre pourriez-vous aller avec sept questions ? avec dix ? Démontrez vos réponses. Puis dites ce qui change si l'adversaire a le droit de mentir une fois.*

La question (c) est le cur de l'affaire, et elle est d'une nature entièrement différente de la (a) : elle ne parle pas de votre méthode, elle parle de toutes les méthodes possibles, y compris celles que personne n'a encore imaginées. C'est ce qu'on appelle une borne inférieure, et savoir en démontrer une est ce qui sépare celui qui programme de celui qui comprend. La variante avec mensonge de la question (d) est un vrai sujet de recherche, posé indépendamment par Alfréd Rényi (1961) et Stanisaw Ulam (1976) ; c'est le point de départ des codes correcteurs d'erreurs, où l'on cherche justement à retrouver la vérité malgré des réponses fausses. DCS-53 vous fera reconnaître la même méthode dans un geste que vous faites depuis toujours.

Références :

- Contexte : Recherche dichotomique — Wikipédia (https://fr.wikipedia.org/wiki/Recherche_dichotomique)
- Contexte : Puissance de deux — Wikipédia (https://fr.wikipedia.org/wiki/Puissance_de_deux)
- Contexte : Le jeu des vingt questions — Wikipédia (en anglais) (https://en.wikipedia.org/wiki/Twenty_questions)

Problème 3 (Chercher un mot dans le dictionnaire — Collège, short). **DCS-53. Problème.** *Décrivez, assez précisément pour qu'un autre puisse l'appliquer sans vous, la méthode que vous employez réellement pour trouver un mot dans un dictionnaire de papier. Démontrez ensuite qu'elle s'arrête toujours — soit en trouvant le mot, soit en établissant qu'il n'y figure pas — et majorez le nombre de fois qu'elle vous fait ouvrir le volume dans un dictionnaire de 60 000 entrées.*

Presque personne ne feuillette un dictionnaire page à page, et presque personne n'a jamais formulé la méthode qu'il emploie à la place : c'est un algorithme que l'on exécute des années durant sans l'avoir écrit une seule fois. Le mettre en mots est l'exercice, et la majoration demandée est la surprise — le nombre est ridiculement petit, et c'est cette petitesse qui explique pourquoi la même idée fait fonctionner les moteurs de recherche. DCS-02 en donne la version formelle, avec invariant de boucle et bogue de débordement à la clé.

Références :

- Contexte : Recherche dichotomique — Wikipédia (https://fr.wikipedia.org/wiki/Recherche_dichotomique)
- Voir aussi : DCS-02, la correction de la recherche dichotomique ([#DCS-02](#))

Problème 4 (Une consigne qu'un robot peut suivre — Collège). **DCS-54. Problème.** *Un exécutant ne comprend que quatre ordres : « avance d'un pas », « tourne d'un quart de tour vers la droite », « pose un jeton », et « répète N fois : ... ». Il n'a aucun bon sens : il fait exactement ce qui est écrit, et rien d'autre.*

- (a) *Écrivez la suite d'ordres qui lui fait parcourir le contour d'un carré de côté quatre pas. Démontrez qu'il revient à sa position de départ et à son orientation de départ.*

- (b) *Écrivez la suite d'ordres qui lui fait poser un jeton à chaque marche d'un escalier de cinq marches, chaque marche mesurant deux pas en avant puis deux pas de côté.*
- (c) *Faites exécuter vos deux textes à la lettre par quelqu'un d'autre, qui a le droit — le devoir — de choisir la lecture la plus bête chaque fois qu'il hésite. Notez chaque endroit où il a pu choisir, et corrigez le texte jusqu'à ce qu'aucun choix ne subsiste.*
- (d) *Une consigne peut être parfaitement claire et fausse quand même. Donnez un exemple de chacun des quatre cas : claire et juste, claire et fausse, ambiguë mais juste par chance, ambiguë et fausse. Dites lequel des quatre est le plus difficile à repérer.*

Seymour Papert a bâti le langage Logo autour de cette idée en 1967 : une tortue qui n'obéit qu'à quelques ordres, et l'enfant qui découvre que « faire un carré » et « dire comment faire un carré » sont deux compétences différentes. Scratch, aujourd'hui, en est l'héritier direct. Le point sérieux, c'est que l'ordinateur est exactement cet exécutant sans bon sens : il ne devine rien, ne corrige rien, et n'a jamais l'intention que vous aviez. La question (c) est la véritable expérience — presque tout le monde écrit d'abord une consigne que seul quelqu'un qui sait déjà pourrait suivre, et la découvrir est plus utile que dix leçons.

Références :

- Contexte : Algorithme — Wikipédia (<https://fr.wikipedia.org/wiki/Algorithme>)
- Contexte : Logo (langage) — Wikipédia ([https://fr.wikipedia.org/wiki/Logo_\(langage\)](https://fr.wikipedia.org/wiki/Logo_(langage)))
- Contexte : Seymour Papert — Wikipédia (https://fr.wikipedia.org/wiki/Seymour_Papert)

Problème 5 (Les cinq cartes qui devinent votre nombre — Collège). ***DCS-55. Problème.** On fabrique cinq cartes portant chacune une liste de nombres entre 1 et 31 : sur la première figurent tous les nombres dont l'écriture comme somme de nombres choisis parmi 1, 2, 4, 8, 16 utilise le 1 ; sur la deuxième, tous ceux dont l'écriture utilise le 2 ; et ainsi de suite. Vous pensez à un nombre, on vous montre les cartes une à une, vous dites seulement sur lesquelles il figure — et l'on annonce votre nombre.*

- (a) *Démontrez que tout entier de 1 à 31 s'écrit d'exactlyement une façon comme somme de nombres choisis parmi 1, 2, 4, 8 et 16. Il y a deux choses à établir : qu'une telle écriture existe, et qu'il n'y en a pas deux.*
- (b) *Déduisez-en pourquoi le tour fonctionne, et dites précisément quel calcul il faut faire une fois connue la liste des cartes où le nombre figure.*
- (c) *Écrivez la première carte en entier. Combien de nombres porte-t-elle ? Démontrez-le sans les compter un à un.*
- (d) *Combien de cartes faudrait-il pour aller jusqu'à 1000 ? Démontrez votre réponse. Et si chaque carte pouvait recevoir trois réponses au lieu de deux, quels nombres écririez-vous à la place de 1, 2, 4, 8, 16 ?*

Leibniz a publié en 1703, dans les mémoires de l'Académie royale des sciences de Paris, une *Explication de l'arithmétique binaire* où il montre que deux chiffres suffisent à écrire tous les nombres ; il y voyait une élégance presque théologique, et n'imaginait évidemment pas les machines qui en viendraient trois siècles plus tard. Le tour de cartes est aujourd'hui l'une des activités les plus répandues de l'informatique « débranchée », qui enseigne les idées de la discipline sans le moindre ordinateur — et il est honnête, en ceci qu'il ne cache aucun truc : le tour marche parce que le théorème de la question (a) est vrai. La variante à trois réponses de la question (d) est un très vieux problème, celui des poids de Bachet de Méziriac (1612), où il s'agit de peser toutes les masses entières avec le moins de poids possible.

Références :

- Contexte : Système binaire — Wikipédia (https://fr.wikipedia.org/wiki/Syst%C3%A8me_binaire)
- Contexte : Claude-Gaspard Bachet de Méziriac — Wikipédia (https://fr.wikipedia.org/wiki/Claude-Gaspard_Bachet_de_M%C3%A9ziriac)
- Cours : CS Unplugged — activités d'informatique sans ordinateur (en anglais) (<https://www.csunplugged.org/>)

Problème 6 (L'algorithme d'Euclide, déroulé à la main — Collège). **DCS-56. Problème.** Pour trouver le plus grand diviseur commun de deux entiers strictement positifs a et b , avec $a > b$, on remplace le couple (a, b) par le couple (b, r) , où r est le reste de la division euclidienne de a par b ; on recommence jusqu'à obtenir un reste nul.

- Déroulez-le à la main sur 1071 et 462, puis sur 2024 et 748, en écrivant chaque division sur une ligne.
- Démontrez le fait sur lequel tout repose : les diviseurs communs de a et de b sont exactement les diviseurs communs de b et de r . Commencez par le cas plus simple où $r = a - b$.
- Démontrez que le procédé s'arrête toujours, quel que soit le couple de départ.
- Rassemblez (b) et (c) en une seule démonstration complète : le dernier reste non nul est bien le plus grand diviseur commun cherché. C'est ce qu'on appelle démontrer la correction d'un algorithme.
- Cherchez, parmi les couples d'entiers inférieurs à 100, celui qui fait faire le plus d'étapes à l'algorithme. Regardez la suite des nombres qui apparaissent au fil du calcul : la reconnaissez-vous ?

C'est le plus vieux algorithme non trivial encore en service : il figure aux propositions 1 et 2 du livre VII des *Éléments* d'Euclide, vers 300 av. J.-C., sous forme de soustractions répétées de longueurs, et il tourne aujourd'hui dans chaque transaction chiffrée que fait votre téléphone. La question (e) touche à un moment historique : en 1844, Gabriel Lamé a démontré que le pire cas de l'algorithme d'Euclide est lié à la suite de Fibonacci — l'un des tout premiers résultats jamais obtenus sur le *temps de calcul* d'une méthode, et non sur son résultat. DCS-30 propose une variante qui se passe entièrement de divisions.

Références :

- Contexte : Algorithme d'Euclide — Wikipédia (https://fr.wikipedia.org/wiki/Algorithme_d%27Euclide)
- Contexte : Plus grand commun diviseur — Wikipédia (https://fr.wikipedia.org/wiki/Plus_grand_commun_diviseur)
- Contexte : Gabriel Lamé — Wikipédia (https://fr.wikipedia.org/wiki/Gabriel_Lam%C3%A9)

Problème 7 (Le bit qui rattrape une erreur — Collège). **DCS-57. Problème.** Disposez 36 cartes en carré 6×6 , chacune montrant au hasard sa face noire ou sa face blanche. Ajoutez une septième carte au bout de chaque ligne, de sorte que chaque ligne de sept cartes contienne un nombre pair de faces noires; puis, de la même façon, une septième carte au bas de chaque colonne. La case du coin est alors réclamée deux fois, une fois par sa ligne et une fois par sa colonne.

- Démontrez que ces deux exigences ne se contredisent jamais : la couleur que réclame la ligne du coin est toujours celle que réclame sa colonne. (Comptez de deux façons le nombre total de faces noires.)
- Pendant que vous avez le dos tourné, quelqu'un retourne exactement une carte. Démontrez que vous pouvez toujours dire laquelle.

- (c) *Il en retourne maintenant exactement deux. Démontrez que vous vous apercevez toujours que quelque chose a changé, puis donnez un exemple où vous ne pouvez pas dire lesquelles.*
- (d) *Et s'il en retournerait quatre, bien choisies ? Décidez, avec démonstration, si vous êtes encore certain de vous apercevoir de quelque chose.*

À la fin des années 1940, Richard Hamming disposait des Bell Labs le week-end pour faire tourner ses calculs sur une machine à relais; au moindre défaut de lecture, la machine s'arrêtait et le week-end était perdu. « Si elle sait qu'il y a une erreur, pourquoi ne saurait-elle pas où ? » — de cette exaspération est sorti le code de Hamming (1950) et, avec lui, toute la théorie des codes correcteurs. Les cartes de ce problème en sont la version la plus simple, et elles vous font traverser en quatre questions tout le sujet : détecter une erreur, la localiser, découvrir la limite du procédé, et voir exactement quelles configurations d'erreurs passent au travers. Le même principe protège vos numéros de carte bancaire, les codes ISBN des livres et les données envoyées par les sondes spatiales.

Références :

- Contexte : Somme de contrôle — Wikipédia (https://fr.wikipedia.org/wiki/Somme_de_contr%C3%B4le)
- Contexte : Code de Hamming — Wikipédia (https://fr.wikipedia.org/wiki/Code_de_Hamming)
- Contexte : Richard Hamming — Wikipédia (https://fr.wikipedia.org/wiki/Richard_Hamming)

Problème 8 (Deux exécutions de la même consigne — Collège, short). ***DCS-58. Problème.** Appliquez à la liste 4, 7, 1 la consigne « prends le premier nombre, ajoute 2 et multiplie par 3, écris le résultat, puis recommence avec le nombre suivant » de deux façons toutes deux défendables et qui ne donnent pas la même sortie. Réécrivez ensuite la consigne pour qu'aucune autre lecture ne soit possible, et démontrez que votre version ne laisse plus le choix.*

Le 23 septembre 1999, la sonde Mars Climate Orbiter s'est perdue parce que deux équipes avaient lu la même grandeur dans deux unités différentes, l'une en newtons, l'autre en livres-force : une mission de plusieurs centaines de millions de dollars détruite par une consigne que l'on pouvait lire de deux façons. Un ordinateur ne demande jamais « tu veux dire quoi ? » — il choisit une lecture, la vôtre ou l'autre, et continue. Écrire sans ambiguïté n'est donc pas une élégance de rédaction mais une exigence technique, et la question difficile de ce problème est la dernière : démontrer qu'un texte ne peut plus être lu de deux manières est beaucoup plus dur que de le croire clair.

Références :

- Contexte : Ambiguïté — Wikipédia (<https://fr.wikipedia.org/wiki/Ambigu%C3%Aft%C3%A9>)
- Contexte : Mars Climate Orbiter — Wikipédia (https://fr.wikipedia.org/wiki/Mars_Climate_Orbiter)
- Contexte : Spécification (informatique) — Wikipédia ([https://fr.wikipedia.org/wiki/Sp%C3%A9cification_\(informatique\)](https://fr.wikipedia.org/wiki/Sp%C3%A9cification_(informatique)))

Problème 9 (Sortir du labyrinthe — Collège). ***DCS-59. Problème.** Vous êtes dans un labyrinthe, sans plan et sans vue d'ensemble. La « règle de la main droite » dit : posez la main droite sur un mur et avancez sans jamais la décrocher.*

- (a) *Dessinez un labyrinthe dont tous les murs forment un seul bloc rattaché au bord extérieur, et suivez-y la règle. Décrivez ce qui se passe, et expliquez pourquoi vous finissez par ressortir.*
- (b) *Dessinez maintenant un labyrinthe où la règle échoue — où l'on tourne indéfiniment sans jamais atteindre le but. Dites ce que votre labyrinthe a que celui de (a) n'avait pas.*

- (c) *Voici une méthode qui, elle, ne se laisse pas piéger. En entrant dans un couloir, tracez un trait à son entrée ; en en sortant, tracez un trait à sa sortie. N'empruntez jamais un couloir portant déjà deux traits ; à un carrefour où vous n'êtes jamais venu, prenez n'importe quel couloir vierge ; à un carrefour déjà visité, revenez sur vos pas si le couloir d'où vous venez ne porte qu'un trait, et sinon prenez un couloir en portant le moins. Démontrez que chaque couloir est parcouru au plus deux fois, une fois dans chaque sens.*
- (d) *Expliquez, avec le plus de précision que vous pourrez, pourquoi cette méthode finit par visiter tout carrefour accessible depuis votre point de départ. Puis désignez l'endroit de votre explication qui s'appuie sur quelque chose que vous n'avez pas démontré.*

La méthode de la question (c) est due à Trémaux, ingénieur français du XIX^e siècle, dont Édouard Lucas rapporte le procédé dans le premier tome de ses *Récréations mathématiques* (1882), où il occupe le chapitre sur les labyrinthes. On la reconnaît aujourd'hui comme le parcours en profondeur, l'un des deux ou trois algorithmes de graphes dont tout le reste dépend — et c'est un fait remarquable qu'un problème posé par un promeneur perdu et un problème posé par un compilateur reçoivent la même réponse. La question (d) est délibérément honnête sur son niveau : elle vous demande de repérer le trou de votre propre démonstration, ce qui est un savoir-faire à part entière et souvent le premier pas vers la bonne démonstration.

Références :

- Contexte : Parcours en profondeur — Wikipédia (https://fr.wikipedia.org/wiki/Parcours_en_profondeur)
- Contexte : Algorithmes de résolution de labyrinthe — Wikipédia (en anglais) (https://en.wikipedia.org/wiki/Maze-solving_algorithm)
- Lecture : É. Lucas, *Récréations mathématiques*, tome I (1882), le chapitre sur les labyrinthes

Problème 10 (Trois cartes, combien de comparaisons ? — Collège, short). *DCS-60. Problème.* *On vous donne trois cartes de valeurs différentes, face cachée, et vous n'avez le droit que de comparer deux cartes à la fois. Démontrez que trois comparaisons suffisent toujours à les ranger dans l'ordre, et qu'aucune méthode n'y parvient à coup sûr en deux.*

La première moitié se règle en exhibant une méthode ; la seconde parle de toutes les méthodes possibles, et se règle en comptant ce que deux réponses peuvent au plus distinguer. C'est la plus petite borne inférieure honnête qui existe, et c'est le même argument qui, poussé jusqu'au bout, démontre qu'aucun tri par comparaisons ne peut être sensiblement plus rapide que ceux qu'on connaît. DCS-29 en donne la version « trouver le plus grand » et DCS-51 la version « ranger sept cartes ».

Références :

- Contexte : Tri par comparaison — Wikipédia (https://fr.wikipedia.org/wiki/Tri_par_comparaison)
- Contexte : Algorithme de tri — Wikipédia (https://fr.wikipedia.org/wiki/Algorithme_de_tri)

Lycée

Problème 11 (La parité : l'échiquier mutilé — Lycée). *DCS-01. Problème.*

- (a) *On retire d'un échiquier ordinaire 8×8 deux cases d'angle diagonalement opposées. Démontrez que les 62 cases restantes ne peuvent pas être pavées par 31 dominos, chacun recouvrant deux cases adjacentes.*

- (b) Démontrez que, dans toute assemblée de personnes, le nombre de celles qui ont serré un nombre impair de mains est pair.
- (c) Énoncez, en une phrase chacun, l'unique invariant qui fait marcher chaque démonstration.

L'échiquier mutilé a été popularisé par Martin Gardner (1957) et, avant lui, par le manuel de logique de Max Black ; le fait sur les poignées de main est d'Euler, tiré de l'article de 1736 sur les ponts de Königsberg qui a fondé la théorie des graphes. La parité est l'invariant non trivial le plus simple des mathématiques, et ces deux problèmes en sont l'initiation canonique : une démonstration d'impossibilité qu'aucune quantité d'essais infructueux ne saurait fournir. John McCarthy proposa plus tard l'échiquier comme banc d'essai du raisonnement automatique, précisément parce que la courte démonstration humaine ne ressemble en rien à une recherche exhaustive.

Références :

- Contexte : Problème de l'échiquier mutilé — Wikipédia (https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_l%27%C3%A9chiquier_mutil%C3%A9)
- Contexte : Lemme des poignées de main — Wikipédia (https://fr.wikipedia.org/wiki/Lemme_des_poign%C3%A9es_de_main)
- Lecture : Lehman, Leighton & Meyer, *Mathematics for Computer Science*, chapitre sur les invariants — gratuit sur MIT OCW 6.042J (<https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-s>)

Problème 12 (Pourquoi la recherche dichotomique est correcte — Lycée). *DCS-02. Problème.* La recherche dichotomique cherche une valeur t dans un tableau trié $a_1 \leq a_2 \leq \dots \leq a_n$ en maintenant deux indices $\ell \leq r$, en comparant t à l'élément du milieu, puis en écartant la moitié qui ne peut pas contenir t .

- (a) Écrivez l'algorithme avec précision, dans un pseudocode de votre cru.
- (b) Énoncez un invariant de boucle — une phrase vraie avant chaque itération — et démontrez par récurrence qu'il se maintient et que l'algorithme répond toujours correctement.
- (c) Démontrez que la boucle s'exécute au plus $\lfloor \log_2 n \rfloor + 1$ fois.
- (d) Expliquez où votre démonstration s'effondre si le tableau n'est pas trié, et trouvez le bogue subtil de la formule « évidente » du milieu $m = (\ell + r)/2$ lorsqu'on l'implémente avec des entiers de taille fixe.

Jon Bentley rapporte dans *Programming Pearls* que, lorsqu'il demandait à des programmeurs professionnels d'écrire une recherche dichotomique, environ quatre-vingt-dix pour cent se trompaient ; le débordement du milieu de la question (d) a survécu des années durant dans la bibliothèque standard de Java. L'idée fut publiée pour la première fois par John Mauchly en 1946 et pourtant, d'après Knuth, la première version publiée correcte a attendu 1962. C'est le plus petit exemple complet, sur ce site, de la discipline qu'enseigne cette page : un algorithme est un théorème, et l'invariant de boucle en est l'hypothèse de récurrence.

Références :

- Contexte : Recherche dichotomique — Wikipédia (https://fr.wikipedia.org/wiki/Recherche_dichotomique)
- Contexte : Invariant de boucle — Wikipédia (https://fr.wikipedia.org/wiki/Invariant_de_boucle)
- Lecture : Bentley, *Programming Pearls*, ch. 4
- Cours : MIT OCW 6.042J Mathematics for Computer Science (<https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/>)

Problème 13 (Dénombrer les chaînes de bits — Lycée). *DCS-03. Problème.* Une chaîne de bits est une suite finie de 0 et de 1.

- (a) Démontrez qu'il y a exactement 2^n chaînes de bits de longueur n , et exactement $\binom{n}{k}$ d'entre elles comportant exactement k uns.
- (b) Soit c_n le nombre de chaînes de bits de longueur n sans deux 1 consécutifs. Calculez $c_1, \dots, c_5$, conjecturez la règle et démontrez que $c_n = c_{n-1} + c_{n-2}$; concluez que c_n est un nombre de Fibonacci.
- (c) Utilisez (a) pour démontrer l'identité $\binom{n}{0} + \binom{n}{1} + \dots + \binom{n}{n} = 2^n$ en dénombrant un même ensemble de deux façons.

Dénombrer des chaînes soumises à des contraintes est la plus ancienne des mathématiques de l'informatique — de plusieurs siècles antérieure aux ordinateurs : le compte de la question (b) apparaît dans la prosodie de l'Inde ancienne (Pingala, puis plus tard Hemachandra, vers 1150), où des syllabes de longueur un et deux devaient paver un vers, anticipant Fibonacci de quelques décennies. Le double dénombrement, technique de la question (c), est l'une des méthodes de démonstration les plus discrètement puissantes de la combinatoire ; Aigner et Ziegler lui consacrent un chapitre dans *Proofs from THE BOOK*.

Références :

- Contexte : Suite de Fibonacci — Wikipédia (https://fr.wikipedia.org/wiki/Suite_de_Fibonacci)
- Contexte : Preuve par double dénombrement — Wikipédia (https://fr.wikipedia.org/wiki/Preuve_par_double_d%C3%A9nombrement)
- Lecture : Lehman, Leighton & Meyer, *Mathematics for Computer Science*, chapitres de dénombrement — gratuit sur MIT OCW 6.042J (<https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science>)

Problème 14 (Chevaliers et valets, formalisés — Lycée). **DCS-04. Problème.** *Sur une île, chaque habitant est un chevalier (qui dit toujours la vérité) ou un valet (qui ment toujours). Vous rencontrez A et B. A dit : « B est un valet. » B dit : « A et moi sommes de la même espèce. » (a) Introduisez des variables propositionnelles ($p = \text{« A est un chevalier »}$, $q = \text{« B est un chevalier »}$), traduisez chaque déclaration en une formule qui doit être vraie exactement lorsque celui qui la prononce est un chevalier, et résolvez le système obtenu par table de vérité. (b) Démontrez que votre réponse est la seule qui soit cohérente. (c) Vous rencontrez maintenant C, qui dit « Je suis un valet. » Démontrez qu'aucune affectation n'est cohérente, et expliquez soigneusement le rapport avec la phrase « cette phrase est fausse ».*

Raymond Smullyan a bâti tout un genre littéraire sur ces énigmes dans *What Is the Name of This Book ?* (1978), et il s'en est servi comme d'un escalier doux menant jusqu'au théorème d'incomplétude de Gödel — la question (c) en est la première marche. Le contenu sérieux, c'est la traduction elle-même : transformer une parole en formules dont la vérité se calcule est exactement ce que fait un compilateur, un vérificateur ou un solveur SAT, et c'est un savoir-faire qu'il vaut mieux acquérir sur des énigmes avant qu'il ne compte vraiment.

Références :

- Contexte : Chevaliers et valets — Wikipédia (en anglais) (https://en.wikipedia.org/wiki/Knights_and_Knaves)
- Contexte : Calcul des propositions — Wikipédia (https://fr.wikipedia.org/wiki/Calcul_des_propositions)
- Lecture : Smullyan, *What Is the Name of This Book ?*
- Cours : MIT OCW 6.042J Mathematics for Computer Science (<https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/>)

Problème 15 (Combien de comparaisons pour trouver le plus grand ? — Lycée, short). **DCS-29. Problème.** *On vous donne n nombres distincts et vous ne pouvez comparer que deux à la fois.*

Démontrez que $n - 1$ comparaisons suffisent pour trouver le plus grand, et — moitié plus difficile — démontrez qu'aucune méthode ne peut toujours y parvenir en moins.

La borne supérieure est une boucle que n'importe qui sait écrire ; la borne inférieure est un véritable argument, et la voie habituelle consiste à voir chaque comparaison comme éliminant exactement un candidat, de sorte qu'il faut $n - 1$ éliminations pour n'en laisser qu'un debout. C'est le plus petit exemple honnête de borne inférieure algorithmique — un énoncé qui ne porte pas sur une méthode mais sur toutes les méthodes possibles — et cette distinction est toute la théorie de la complexité en miniature. Suite naturelle : trouver le plus grand *et* le deuxième plus grand demande environ $n + \log_2 n$ comparaisons, et non $2n$.

Références :

- Contexte : Algorithme de sélection — Wikipédia (https://fr.wikipedia.org/wiki/Algorithme_de_s%C3%A9lection)
- Lecture : Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, ch. 9
- Voir aussi : Combinatoire et théorie des graphes ([pure-combinatorics.html](https://www.pure-combinatorics.html)) pour le lemme des poignées de main

Problème 16 (Un PGCD sans division — Lycée, short). *DCS-30. Problème.* Pour des entiers strictement positifs a, b , démontrez les trois identités

$$\gcd(2a, 2b) = 2 \gcd(a, b), \quad \gcd(2a, b) = \gcd(a, b) \text{ pour } b \text{ impair}, \quad \gcd(a, b) = \gcd(a-b, b) \text{ pour } a > b.$$

Déduisez-en que ces seules règles calculent n'importe quel PGCD à l'aide de rien d'autre que des comparaisons, des soustractions et des divisions par deux, et démontrez que le procédé se termine toujours.

Ces trois règles constituent l'algorithme binaire du PGCD, publié sous sa forme moderne par le physicien Josef Stein en 1967, mais déjà reconnaissable dans les *Neuf Chapitres sur l'art mathématique* de la Chine des Han : « si l'on peut, prendre la moitié ; sinon, poser le dénominateur et le numérateur, retrancher le plus petit du plus grand, et faire cela alternativement jusqu'à les rendre égaux. » La division coûte cher en matériel et la division par deux est gratuite en binaire : cette recette de simplification de fractions vieille de deux mille ans reste donc compétitive face à l'algorithme d'Euclide (DCS-11) au cur des bibliothèques arithmétiques réelles.

Références :

- Contexte : Algorithme binaire de calcul du PGCD — Wikipédia (https://fr.wikipedia.org/wiki/Algorithme_binaire_de_calcul_du_PGCD)
- Lecture : Knuth, *The Art of Computer Programming*, vol. 2, §4.5.2

Problème 17 (Le vote majoritaire en un seul passage — Lycée). *DCS-31. Problème.* Appelons élément majoritaire de la liste $a_1, \dots, a_n$ une valeur qui y apparaît plus de $n/2$ fois. Parcourez la liste en ne conservant qu'un candidat c et un compteur k , en partant de $k = 0$: à la lecture de a_i , si $k = 0$ posez $c = a_i$ et $k = 1$; sinon, si $a_i = c$ augmentez k de un ; sinon diminuez k de un.

- (a) Déroulez le parcours à la main sur 3, 3, 4, 2, 4, 4, 2, 4, 4 puis sur 1, 2, 3, 4, 5, en notant (c, k) après chaque étape.
- (b) Démontrez l'invariant qui porte l'algorithme : après lecture des i premières entrées, celles-ci se décomposent en k copies de c et en paires d'entrées dont les deux membres diffèrent.
- (c) Déduisez-en que si la liste possède un élément majoritaire, le c final est cet élément.

- (d) Montrez par un exemple que le c final peut ne pas être majoritaire lorsqu'aucun élément ne l'est, de sorte qu'un second passage comptant les occurrences de c est vraiment nécessaire. Concluez que la majorité se décide en temps $O(n)$ avec $O(1)$ mémoire au-delà de l'entrée.

Robert Boyer et J Strother Moore ont trouvé cela en 1980 au SRI International en réfléchissant aux systèmes tolérants aux pannes, où un « vote » entre calculs redondants doit se tenir vite et presque sans mémoire ; ils l'ont fait circuler comme rapport technique et ne l'ont publié qu'en 1991, dans un volume en l'honneur de Woody Bledsoe. Le même tandem avait déjà donné au monde l'algorithme de recherche de chaîne de Boyer–Moore et le démonstrateur automatique de Boyer–Moore, et ils ont traité cet algorithme comme un cas d'essai pour la vérification mécanique — destin bien choisi, car sa correction reste entièrement invisible tant qu'on n'a pas trouvé l'invariant de la question (b), et devient parfaitement évidente ensuite. Misra et Gries l'ont généralisé en 1982 pour trouver tous les éléments apparaissant plus de n/k fois.

Références :

- Contexte : Algorithme de vote majoritaire de Boyer–Moore — Wikipédia (en anglais) (https://en.wikipedia.org/wiki/Boyer%E2%80%93Moore_majority_vote_algorithm)
- Article : Boyer & Moore, « MJRTY — A Fast Majority Vote Algorithm », dans *Automated Reasoning : Essays in Honor of Woody Bledsoe* (1991)
- Article : Misra & Gries, « Finding repeated elements », *Science of Computer Programming* 2 (1982)

Problème 18 (La location de skis : décider sans connaître l'avenir — Lycée). *DCS-41. Problème.* Vous skiez pendant un nombre de jours que vous ne connaîtrez qu'à la fin de la saison, laquelle s'achèvera sans prévenir. Louer des skis coûte 1 par jour ; les acheter coûte b une fois pour toutes, après quoi skier est gratuit. Chaque matin vous devez décider, en sachant seulement combien de jours vous avez déjà skié. Appelons une stratégie c -compétitive si, sur toute saison possible, son coût total vaut au plus c fois le coût de la meilleure stratégie qui aurait connu d'avance la durée de la saison.

- (a) Montrez que la meilleure stratégie sachant que la saison dure n jours paie $\min(n, b)$.
- (b) Démontrez que la stratégie « louer les jours 1 à $b - 1$, acheter le jour b » est $(2 - \frac{1}{b})$ -compétitive.
- (c) Démontrez qu'aucune stratégie déterministe n'atteint un rapport inférieur à $2 - \frac{1}{b}$: quel que soit le jour où le skieur a décidé d'acheter, un adversaire qui met fin à la saison au bon moment lui impose ce rapport.
- (d) Expliquez en un paragraphe pourquoi (c) est un énoncé sur toute stratégie concevable et non sur celle de (b), et décrivez ce qui change si la durée de la saison est au contraire tirée selon une loi de probabilité que vous connaissez.

Ce petit dilemme est entré en informatique par le matériel : l'article de 1988 de Karlin, Manasse, Rudolph et Sleator sur le « competitive snoopy caching » traitait d'un cache multiprocesseur devant choisir entre payer indéfiniment un petit coût par accès distant ou payer une fois pour prendre possession d'un bloc mémoire ; et la notion de rapport de compétitivité — qui mesure un algorithme en ligne contre un algorithme hors ligne omniscient, introduite par Sleator et Tarjan en 1985 pour la pagination — est devenue la façon standard de donner un sens au pire cas des décisions prises sans l'avenir. La version aléatoire, où le skieur lance des pièces que l'adversaire ne voit pas, atteint le rapport strictement meilleur $e/(e - 1) \approx 1,58$; cet écart est l'une des démonstrations les plus nettes qui soient de ce que le hasard aide véritablement contre un adversaire. Le même calcul est refait chaque jour par quiconque arbitre entre instances cloud à la demande et instances réservées, et par la pile TCP décidant combien de temps retarder un accusé de réception.

Références :

- Contexte : Problème de la location de skis — Wikipédia (https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_la_location_de_skis)
- Contexte : Analyse compétitive (algorithme en ligne) — Wikipédia (en anglais) ([https://en.wikipedia.org/wiki/Competitive_analysis_\(online_algorithm\)](https://en.wikipedia.org/wiki/Competitive_analysis_(online_algorithm)))
- Article : Karlin, Manasse, Rudolph & Sleator, « Competitive snoopy caching », *Algorithmica* 3 (1988)
- Lecture : Borodin & El-Yaniv, *Online Computation and Competitive Analysis*, ch. 1

Problème 19 (Les rotations se cachent dans une chaîne doublée — Lycée, short). **DCS-42. Problème.** Soient x et y deux chaînes de même longueur n sur un alphabet quelconque. Démontrez que x est une rotation circulaire de y si et seulement si x apparaît comme facteur contigu de la concaténation yy , et déduisez-en que la rotation se teste en temps $O(n)$ dès qu'on dispose d'une recherche de facteur en temps linéaire.

Un sens est un calcul d'indices qui tient en une ligne ; l'autre demande du soin, et le faire honnêtement, c'est dire exactement quelles positions d'occurrence dans yy sont possibles et pourquoi. Au-delà de l'astuce, la leçon est celle de la réduction : plutôt que d'inventer un algorithme pour un problème nouveau, on transforme l'entrée de sorte qu'un algorithme auquel on fait déjà confiance — ici la recherche de Knuth–Morris–Pratt de DCS-44 — y réponde à notre place. C'est le même geste, à une échelle infiniment plus petite, qui transforme un problème NP-complet en mille autres dans DCS-20.

Références :

- Contexte : Algorithme de recherche de sous-chaîne — Wikipédia (https://fr.wikipedia.org/wiki/Algorithme_de_recherche_de_sous-cha%C3%A9ne)
- Contexte : Algorithme de Knuth–Morris–Pratt — Wikipédia (https://fr.wikipedia.org/wiki/Algorithme_de_Knuth-Morris-Pratt)

Licence

Problème 20 (Ordonner les vitesses de croissance — Licence). **DCS-05. Problème.** Écrivons $f \prec g$ lorsque $f(n)/g(n) \rightarrow 0$. (a) Rangez les fonctions suivantes en une seule chaîne pour $\prec$, en démontrant chaque comparaison consécutive à partir de la définition :

$$\log_2 n, \quad \sqrt{n}, \quad \frac{n}{\log_2 n}, \quad n, \quad n \log_2 n, \quad n^{\log_2 3}, \quad n^2, \quad n^{10}, \quad 1,01^n, \quad 2^n, \quad n!, \quad n^n.$$

(b) Démontrez les deux lemmes généraux qui font l'essentiel du travail : $\log n \prec n^\varepsilon$ pour tout $\varepsilon > 0$, et $n^k \prec c^n$ pour tout k et tout $c > 1$. (c) Démontrez $2^n \prec n!$ et $n! \prec n^n$, et montrez que $\log_2(n!)$ croît comme $n \log_2 n$ à un facteur constant près.

Paul Bachmann a introduit la notation O en 1894 et Edmund Landau l'a diffusée ; Knuth a fait campagne dans les années 1970 pour en fixer l'usage informatique. La chaîne ci-dessus est la règle graduée du domaine : qui ne sait pas situer instantanément $n \log n$ par rapport à $n^{1,01}$ navigue sans instrument. Les comparaisons ont l'air évidentes et le sont pour la plupart — raison précise pour laquelle il vaut la peine de les démontrer une fois à partir de la définition, là où $n^{\log_2 3}$ (l'exposant de Karatsuba, voir DCS-06) demande, lui, un vrai moment de réflexion.

Références :

- Contexte : Comparaison asymptotique (notation de Landau) — Wikipédia (https://fr.wikipedia.org/wiki/Comparaison_asymptotique)

- Lecture : Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, ch. 3
- Lecture : Graham, Knuth & Patashnik, *Concrete Mathematics*, ch. 9

Problème 21 (Récurrences et théorème maître — Licence). **DCS-06. Problème.** *Les temps d'exécution des algorithmes « diviser pour régner » vérifient $T(n) = aT(n/b) + \Theta(n^c)$ avec $a \geq 1$, $b > 1$. Le théorème maître affirme que $T(n)$ vaut $\Theta(n^c)$ si $c > \log_b a$, $\Theta(n^c \log n)$ si $c = \log_b a$, et $\Theta(n^{\log_b a})$ si $c < \log_b a$.*

- Appliquez-le au tri fusion ($a = b = 2, c = 1$), à la multiplication de Karatsuba ($a = 3, b = 2, c = 1$) et à la multiplication matricielle de Strassen ($a = 7, b = 2, c = 2$), en énonçant chaque conclusion.*
- Démontrez le théorème pour n puissance de b en déroulant l'arbre de récursion : summez le travail par niveau comme une série géométrique et traitez les trois régimes.*
- Exhibez une récurrence naturelle que le théorème ne couvre pas, et résolvez-la quand même.*

L'image de l'arbre de récursion n'est de personne et de tout le monde ; le « théorème maître » sous forme empaquetée a été popularisé par la première édition de CLRS (1990), avec des racines dans un article de 1980 de Bentley, Haken et Saxe. Les trois régimes sont une leçon sur le lieu où réside le travail — les feuilles, chaque niveau, ou la racine — et les deux applications de la question (a) sont historiques : l'algorithme de Karatsuba de 1960, trouvé en une semaine après que Kolmogorov eut conjecturé en séminaire que n^2 était optimal, et le coup de tonnerre de Strassen en 1969, montrant que l'élimination de Gauss ne l'est pas davantage. Tous deux ont brisé des bornes inférieures « évidentes » en refusant de calculer de la façon évidente ; la seconde histoire se poursuit en DCS-28.

Références :

- Contexte : Master theorem (analyse d'algorithmes) — Wikipédia (https://fr.wikipedia.org/wiki/Master_theorem)
- Contexte : Algorithme de Karatsuba — Wikipédia (https://fr.wikipedia.org/wiki/Algorithme_de_Karatsuba)
- Lecture : Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, ch. 4
- Cours : MIT OCW 6.006 Introduction to Algorithms (<https://ocw.mit.edu/courses/6-006-introduction-to-algorithms/>)

Problème 22 (L'algorithme de Dijkstra, et où loge la positivité — Licence). **DCS-07. Problème.** *L'algorithme de Dijkstra calcule les plus courts chemins depuis une source s dans un graphe dont les arêtes portent des poids $w \geq 0$: on retire à répétition de la frontière le sommet non visité u de plus petite distance provisoire $d[u]$, on le marque définitif, et on relâche ses arcs sortants.*

- Énoncez et démontrez l'invariant clé : lorsque u est marqué définitif, $d[u]$ est égal à la vraie distance de plus court chemin $\delta(s, u)$.*
- Dans votre démonstration, désignez l'inégalité exacte qui tombe en défaut si un poids d'arête est négatif — une seule ligne.*
- Construisez un graphe avec une seule arête négative (et sans cycle négatif) sur lequel l'algorithme rend une réponse fausse.*
- Expliquez pourquoi les cycles négatifs rendent le problème mal posé, quel que soit l'algorithme.*

Edsger Dijkstra a inventé l'algorithme en 1956 en une vingtaine de minutes, à la terrasse d'un café d'Amsterdam, sans crayon ni papier — contrainte à laquelle il attribuait plus tard sa simplicité — et l'a publié en 1959 dans un article de trois pages. La démonstration gloutonne est le modèle de toute une famille d'arguments dits « de propriété de coupe » (Prim, A*), et la question (b) est

le cur de l'exercice : connaître un algorithme, c'est savoir exactement quelle hypothèse consomme chaque ligne de sa démonstration. L'algorithme plus lent de Bellman et Ford existe précisément parce que (b) échoue.

Références :

- Contexte : Algorithme de Dijkstra — Wikipédia (https://fr.wikipedia.org/wiki/Algorithme_de_Dijkstra)
- Article : Dijkstra, « A note on two problems in connexion with graphs », *Numerische Mathematik* 1 (1959)
- Lecture : Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, ch. 22 (3e éd. : ch. 24)
- Cours : MIT OCW 6.006 Introduction to Algorithms (<https://ocw.mit.edu/courses/6-006-introduction-to-algorithms/>)

Problème 23 (Échange glouton : l'ordonnancement d'intervalles — Licence). *DCS-08. Problème.* Étant donné n requêtes, chacune un intervalle $[s_i, f_i)$, sélectionnez le plus grand nombre possible d'intervalles deux à deux disjoints.

- (a) Démontrez qu'en choisissant à répétition la requête compatible dont la date de fin est la plus précoce on obtient l'optimum, par un argument d'échange (« le glouton reste devant ») : montrez que le k -ième choix glouton se termine au plus tard en même temps que le k -ième intervalle de toute solution optimale.
- (b) Montrez par des contre-exemples que trois rivaux naturelles échouent : la date de début la plus précoce, l'intervalle le plus court, et le moins de conflits.
- (c) Pondérez maintenant chaque intervalle et demandez le poids total maximal ; démontrez que le glouton échoue et expliquez en une phrase quel trait structurel du problème pondéré appelle plutôt la programmation dynamique.

L'ordonnancement d'intervalles est le premier spécimen standard d'algorithme glouton correct, et l'argument d'échange de la question (a) est une technique, non une astuce — on le retrouve dans le codage de Huffman (DCS-14), dans les arbres couvrants de poids minimal et dans la théorie des matroïdes, qui est la réponse générale à « quand le glouton fonctionne-t-il ? ». C'est pour cette raison que Kleinberg et Tardos ouvrent leur manuel avec ce problème. La question (b) enseigne un savoir-faire tout aussi important : tuer vite des algorithmes plausibles.

Références :

- Contexte : Ordonnancement d'intervalles — Wikipédia (en anglais) (https://en.wikipedia.org/wiki/Interval_scheduling)
- Contexte : Algorithme glouton — Wikipédia (https://fr.wikipedia.org/wiki/Algorithme_glouton)
- Lecture : Kleinberg & Tardos, *Algorithm Design*, ch. 4

Problème 24 (La plus longue sous-suite croissante — Licence). *DCS-09. Problème.* Étant donnée une suite de n nombres distincts, une sous-suite croissante choisit des positions $i_1 < i_2 < \dots$ dont les valeurs croissent.

- (a) Posez $L[i]$ = longueur de la plus longue sous-suite croissante se terminant à la position i ; démontrez que la récurrence $L[i] = 1 + \max\{L[j] : j < i, a_j < a_i\}$ (le max de l'ensemble vide valant 0) est correcte, ce qui donne un algorithme en $O(n^2)$.
- (b) Démontrez la correction du raffinement en $O(n \log n)$: maintenez, pour chaque longueur ℓ , la plus petite valeur finale possible d'une sous-suite croissante de longueur ℓ ; montrez que ce tableau reste trié et que chaque élément est traité par une seule recherche dichotomique.
- (c) Déduisez-en le théorème d'Erds-Szekeres : toute suite de $mn + 1$ nombres distincts contient une sous-suite croissante de longueur $m + 1$ ou une sous-suite décroissante de longueur $n + 1$.

La plus longue sous-suite croissante est la programmation dynamique en miniature — une sous-structure optimale que l'on peut démontrer et non seulement croire — et sa solution en $O(n \log n)$, apparentée au jeu de cartes de la patience (Mallows, années 1960), ouvre sur une réelle profondeur : la longueur espérée pour une permutation aléatoire vaut $2\sqrt{n}$, théorème difficile de Vershik–Kerov et Logan–Shepp dont les fluctuations se sont révélées obéir à la loi de Tracy–Widom de la théorie des matrices aléatoires. La question (c), tirée de l'article « happy ending » d'Erds et Szekeres de 1935, montre un principe des tiroirs de deux lignes caché dans la table de programmation dynamique.

Références :

- Contexte : Plus longue sous-suite strictement croissante — Wikipédia (https://fr.wikipedia.org/wiki/Plus_longue_sous-suite_strictement_croissante)
- Contexte : Théorème d'Erds–Szekeres — Wikipédia (https://fr.wikipedia.org/wiki/Th%C3%A9or%C3%A8me_d%27Erds%27Szekeres)
- Lecture : Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, chapitre de programmation dynamique

Problème 25 (La distance d'édition — Licence). **DCS-10. Problème.** *La distance d'édition entre deux chaînes x et y est le nombre minimal d'insertions, de suppressions et de substitutions d'un seul caractère transformant x en y . Soit $D[i][j]$ la distance d'édition entre les i premiers caractères de x et les j premiers de y .*

(a) *Démontrez la récurrence*

$$D[i][j] = \min(D[i-1][j] + 1, D[i][j-1] + 1, D[i-1][j-1] + [x_i \neq y_j])$$

avec les bonnes conditions au bord, en prenant garde à la seule étape non triviale : pourquoi peut-on supposer qu'un script d'édition optimal touche les derniers caractères de l'une exactement de ces trois façons.

(b) *Concluez à un algorithme en temps $O(mn)$, puis en espace $O(\min(m, n))$.*

(c) *Démontrez que la distance d'édition est une distance sur les chaînes.*

Vladimir Levenshtein a introduit cette distance en 1965 en théorie des codes ; le programme dynamique de Wagner et Fischer, en 1974, en a fait le cheval de trait des correcteurs orthographiques, des outils de diff et de la biologie computationnelle (Needleman–Wunsch en est le cousin à score). Un résultat de 2015 de Backurs et Indyk fournit le post-scriptum moderne : un algorithme fortement sous-quadratique réfuterait l'hypothèse forte du temps exponentiel (voir DCS-25) — de sorte que la vénérable table en $O(mn)$ est, plausiblement, essentiellement optimale.

Références :

- Contexte : Distance d'édition — Wikipédia (https://fr.wikipedia.org/wiki/Distance_d%27%C3%A9dition)
- Contexte : Distance de Levenshtein — Wikipédia (https://fr.wikipedia.org/wiki/Distance_de_Levenshtein)
- Lecture : Kleinberg & Tardos, *Algorithm Design*, ch. 6

Problème 26 (L'algorithme d'Euclide et le théorème de Lamé — Licence). **DCS-11. Problème.** *L'algorithme d'Euclide calcule $\gcd(a, b)$ par divisions euclidiennes successives.*

(a) *Démontrez sa correction : $\gcd(a, b) = \gcd(b, a \bmod b)$.*

(b) *Démontrez que les nombres de Fibonacci consécutifs constituent le pire cas : appliqué à F_{n+2}, F_{n+1} , l'algorithme effectue exactement n divisions.*

- (c) Démontrez le théorème de Lamé : si l'algorithme appliqué à $a > b \geq 1$ prend n étapes, alors $b \geq F_{n+1}$; concluez que le nombre d'étapes vaut au plus $\log_{\varphi} b + O(1)$, où φ est le nombre d'or — donc logarithmique en la taille de l'entrée.
- (d) Où exactement la récurrence de Fibonacci entre-t-elle dans la récurrence de la question (c) ?

L'argument de Gabriel Lamé, en 1844, est souvent appelé la première analyse de complexité d'un algorithme — un théorème sur le temps d'exécution dans le pire cas un siècle avant les ordinateurs — et c'est aussi, délicieusement, la première application pratique des nombres de Fibonacci. L'algorithme lui-même, au Livre VII des *Éléments* d'Euclide (vers 300 av. J.-C.), est à certaines mesures le plus ancien algorithme encore en usage quotidien en production : tout échange de clés RSA l'exécute. Knuth lui consacre cinquante pages amoureuses dans le volume 2.

Références :

- Contexte : Algorithme d'Euclide — Wikipédia (https://fr.wikipedia.org/wiki/Algorithme_d%27Euclide)
- Lecture : Knuth, *The Art of Computer Programming*, vol. 2, §4.5.3
- Lecture : Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, chapitre d'algorithmique arithmétique

Problème 27 (Exponentiation rapide et chaînes d'additions — Licence). *DCS-12. Problème.*

- (a) Démontrez que les élévations au carré répétées calculent a^n en au plus $2\lceil \log_2 n \rceil$ multiplications, et donnez la version modulaire calculant $a^n \bmod m$ sans qu'aucune valeur intermédiaire ne dépasse m^2 .
- (b) Une chaîne d'additions pour n est une suite $1 = c_0, c_1, \dots, c_k = n$ dont chaque terme est la somme de deux termes antérieurs (éventuellement égaux) ; les multiplications servant à calculer a^n correspondent aux pas de la chaîne. Démontrez que toute chaîne d'additions pour n a longueur au moins $\log_2 n$.
- (c) Montrez que pour $n = 15$ la méthode binaire utilise 6 multiplications alors qu'il existe une chaîne d'additions de longueur 5 — les carrés répétés ne sont donc pas toujours optimaux.

L'exponentiation rapide apparaît dans la prosodie de Pingala (avant 200 av. J.-C.) et chez al-Kashi ; elle est aujourd'hui le cur battant de RSA et de Diffie–Hellman, exécutée des milliards de fois par jour, toujours modulo m comme en (a). La question d'optimalité de (b)–(c) — le problème des chaînes d'additions de Scholz — est réellement difficile : calculer les chaînes les plus courtes est réputé intraitable, et le volume 2 de Knuth le traite comme un terrain vague ouvert et chéri. Toute une littérature d'ingénierie cryptographique descend de la question (c).

Références :

- Contexte : Exponentiation rapide — Wikipédia (https://fr.wikipedia.org/wiki/Exponentiation_rapide)
- Contexte : Chaîne d'additions — Wikipédia (https://fr.wikipedia.org/wiki/Cha%C3%A9ne_d%27additions)
- Lecture : Knuth, *The Art of Computer Programming*, vol. 2, §4.6.3

Problème 28 (Hachage et borne des anniversaires — Licence). *DCS-13. Problème.* On jette m objets, indépendamment et uniformément, dans n casiers.

- (a) Démontrez que la probabilité que les m objets atterrissent dans des casiers distincts vaut $\prod_{i=1}^{m-1} (1 - i/n)$ et, à l'aide de $1 - x \leq e^{-x}$, montrez qu'elle est au plus $e^{-m(m-1)/(2n)}$.
- (b) Concluez à la borne des anniversaires : les collisions deviennent probables (probabilité $\geq 1/2$) dès que m avoisine $1,18\sqrt{n}$, et évaluez pour les anniversaires ($n = 365$).

- (c) Démontrez une borne inférieure de sens contraire : pour $m \leq \sqrt{2n}$, la probabilité de collision vaut au plus $m^2/(2n)$.
- (d) Tirez les deux conclusions d'ingénierie, chiffres à l'appui : jusqu'à quel remplissage une table de hachage à n cases peut aller avant que les collisions ne dominent, et pourquoi une empreinte de 128 bits n'offre qu'environ 2^{64} de sécurité contre les attaques par collision.

Le paradoxe des anniversaires a été analysé par Richard von Mises en 1939 et c'est ce fait probabiliste rare que tout programmeur et tout cryptographe en exercice doit s'être approprié : les tables de hachage (inventées chez IBM dans les années 1950 par Luhn et Amdahl, entre autres) vivent sous le seuil, et les attaques des anniversaires — la raison pour laquelle les 128 bits de MD5 ont cessé de suffire — vivent au-dessus. Le calcul tient en trois lignes ; l'important est de le posséder, constantes comprises, et non de l'avoir vu une fois.

Références :

- Contexte : Paradoxe des anniversaires — Wikipédia (https://fr.wikipedia.org/wiki/Paradoxe_des_anniversaires)
- Contexte : Attaque des anniversaires — Wikipédia (https://fr.wikipedia.org/wiki/Attaque_des_anniversaires)
- Lecture : Mitzenmacher & Upfal, *Probability and Computing*, ch. 5
- Cours : MIT OCW 6.006 Introduction to Algorithms (<https://ocw.mit.edu/courses/6-006-introduction-to-algorithms>)

Problème 29 (Les codes de Huffman sont optimaux — Licence). *DCS-14. Problème.* Un code préfixe attribue à chaque symbole un mot de code binaire, aucun n'étant préfixe d'un autre ; pour des fréquences de symboles $f_1, \dots, f_n$, son coût vaut $\sum_i f_i \cdot (\text{profondeur du symbole } i)$ dans l'arbre du code. L'algorithme de Huffman fusionne à répétition les deux symboles les moins fréquents en un seul. Démontrez que sa sortie est de coût minimal parmi tous les codes préfixes, en deux temps :

- (a) le lemme d'échange — un arbre optimal admet les deux symboles les moins fréquents comme feuilles surs de profondeur maximale ;
- (b) la récurrence — un arbre optimal pour l'alphabet fusionné s'étend en un arbre optimal pour l'alphabet d'origine.
- (c) Exhibez des fréquences pour lesquelles l'arbre de Huffman n'est pas unique et calculez les deux optima.

David Huffman a trouvé l'algorithme en 1951, étudiant de troisième cycle au MIT, ayant préféré un mémoire à l'examen final du cours de théorie de l'information de Fano ; sa fusion ascendante a battu la méthode descendante de Shannon–Fano que son professeur avait co-inventée, et il faillit jeter l'idée avant d'en reconnaître la valeur. La démonstration est l'argument modèle « échange puis récurrence » de l'optimalité gloutonne, et l'algorithme tourne encore dans JPEG, MP3 et DEFLATE — un mémoire d'étudiant au parcours de déploiement inhabituel.

Références :

- Contexte : Codage de Huffman — Wikipédia (https://fr.wikipedia.org/wiki/Codage_de_Huffman)
- Article : Huffman, « A Method for the Construction of Minimum-Redundancy Codes », *Proceedings of the IRE* 40 (1952)
- Lecture : Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, chapitre sur les algorithmes gloutons

Problème 30 (Le lemme de l'étoile — Licence). *DCS-15. Problème.* (a) Démontrez le lemme de l'étoile pour les langages réguliers : si L est reconnu par un automate fini à p états, alors tout $w \in L$ avec $|w| \geq p$ se factorise en $w = xyz$ avec $|xy| \leq p$, y non vide, et $xy^kz \in L$ pour tout $k \geq 0$.

(Le moteur est le principe des tiroirs appliqué aux états ; dites exactement où.) (b) Servez-vous-en pour démontrer que $\{0^n 1^n : n \geq 0\}$ n'est pas régulier. (c) Démontrez que $\{1^p : p \text{ premier}\}$ n'est pas régulier. (d) Exhibez un langage non régulier qui satisfait pourtant la conclusion du lemme de l'étoile — le lemme est donc une arme, non une caractérisation.

Le lemme est dû à l'article fondateur de Rabin et Scott sur les automates finis (1959), pour lequel ils ont reçu le prix Turing, et il a été affiné par Bar-Hillel, Perles et Shamir en 1961. C'est, pour la plupart des étudiants, la première démonstration d'impossibilité portant sur le calcul : une mémoire finie ne peut, on le démontre, pas compter. La question (d) est la vaccination standard contre la surinterprétation — c'est le théorème de Myhill–Nerode, et non le lemme de l'étoile, qui caractérise exactement la régularité.

Références :

- Contexte : Lemme de l'étoile (langages rationnels) — Wikipédia (https://fr.wikipedia.org/wiki/Lemme_de_l%27%C3%A9toile)
- Lecture : Sipser, *Introduction to the Theory of Computation*, ch. 1
- Cours : MIT OCW 18.404J Theory of Computation (Sipser) (<https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/>)

Problème 31 (Hamming(7,4) : le construire et le démontrer — Licence). **DCS-16. Problème.** Construisez le code de Hamming(7,4) : encodez quatre bits de message en sept en ajoutant trois bits de parité, chacun couvrant un sous-ensemble choisi de positions.

- (a) Donnez explicitement la construction (une matrice génératrice et une matrice de contrôle sur $\mathbb{F}_2$) et démontrez que le code est linéaire, à $2^4 = 16$ mots de code.
- (b) Démontrez que sa distance minimale vaut exactement 3, et qu'une distance minimale de 3 équivaut à la correction de tout basculement d'un seul bit.
- (c) Concevez le décodeur : montrez que les trois contrôles de parité d'un mot reçu épellent, en binaire, la position exacte d'une erreur unique.
- (d) Démontrez que le code est parfait : les 16 boules de Hamming de rayon 1 centrées sur les mots de code pavent tout $\{0, 1\}^7$, puisque $16 \times 8 = 128 = 2^7$.

Richard Hamming a construit ce code aux Bell Labs après des week-ends de frustration : l'ordinateur à relais détectait les erreurs de ses données d'entrée et se contentait de jeter ses travaux, et il s'est demandé pourquoi une machine capable de trouver une erreur ne pouvait pas la corriger. Son article de 1950 a fondé la théorie des codes ; le code (7,4) et ses parents ont ensuite protégé les commutateurs téléphoniques, la télémessure spatiale, la mémoire vive (mémoire ECC) et les codes QR. Le décompte d'empilement de sphères de la question (d) est la porte d'entrée d'une belle théorie — les codes parfaits sont si rares qu'ils ont été entièrement classifiés.

Références :

- Contexte : Code de Hamming (7,4) — Wikipédia ([https://fr.wikipedia.org/wiki/Code_de_Hamming_\(7,4\)](https://fr.wikipedia.org/wiki/Code_de_Hamming_(7,4)))
- Contexte : Code de Hamming — Wikipédia (https://fr.wikipedia.org/wiki/Code_de_Hamming)
- Article : Hamming, « Error detecting and error correcting codes », Bell System Technical Journal 29 (1950)
- Lecture : MacWilliams & Sloane, *The Theory of Error-Correcting Codes*, ch. 1

Problème 32 (Pourquoi doubler est la bonne idée — Licence, short). **DCS-32. Problème.** Un tableau dynamique range ses entrées dans un bloc de capacité fixe ; un ajout coûte une unité et, lorsque le bloc est plein, on alloue un bloc de capacité double et on recopie chaque entrée, au prix

d'une unité par entrée recopiée. Démontrez qu'en partant d'un tableau vide de capacité un, le coût total de n ajouts consécutifs est inférieur à $3n$, et démontrez que cette borne linéaire tombe en défaut si la capacité croît au contraire d'une constante additive fixe.

C'est le premier exercice standard d'analyse amortie — la discipline comptable que Robert Tarjan a nommée et systématisée en 1985, où l'on facture à chaque opération bon marché un petit supplément afin de payer d'avance la rare opération coûteuse. La conclusion explique pourquoi tout tableau extensible des langages modernes (`vector` en C++, `ArrayList` en Java, `list` en Python) multiplie sa capacité au lieu de lui ajouter une constante; le choix du multiplicateur — 2, ou 1,5, ou le nombre d'or — reste discuté entre implémenteurs pour des raisons de réutilisation de la mémoire, mais tout choix supérieur à 1 donne la borne que l'on vous demande de démontrer.

Références :

- Contexte : Tableau dynamique (vecteur) — Wikipédia ([https://fr.wikipedia.org/wiki/Vecteur_\(structure_de_donn%C3%A9es\)](https://fr.wikipedia.org/wiki/Vecteur_(structure_de_donn%C3%A9es)))
- Contexte : Analyse amortie — Wikipédia (https://fr.wikipedia.org/wiki/Analyse_amortie)
- Article : Tarjan, « Amortized computational complexity », SIAM Journal on Algebraic and Discrete Methods 6 (1985)

Problème 33 (Un ordre topologique existe exactement lorsqu'il n'y a pas de cycle — Licence, short). **DCS-33. Problème.** *Un tri topologique d'un graphe orienté fini est un ordre linéaire de ses sommets dans lequel toute arête pointe vers l'avant; démontrez qu'un tel ordre existe si et seulement si le graphe n'a pas de cycle orienté. Donnez ensuite un algorithme produisant un tel ordre en temps $O(|V| + |E|)$ et démontrez sa correction.*

A. B. Kahn a publié en 1962 l'algorithme consistant à retirer à répétition une source, dans un article portant sur l'ordonnancement des tâches d'un grand projet d'ingénierie — le problème d'ordonnancement PERT/CPM qui a motivé une bonne part de l'analyse de réseaux des débuts; la démonstration alternative, qui renverse l'ordre de fin d'un parcours en profondeur, est due à Tarjan en 1976. Le théorème explique pourquoi les systèmes de compilation, le recalcul des tableurs et les gestionnaires de paquets peuvent fonctionner, et pourquoi chacun d'eux doit signaler une erreur quand vous écrivez une dépendance circulaire.

Références :

- Contexte : Tri topologique — Wikipédia (https://fr.wikipedia.org/wiki/Tri_topologique)
- Lecture : Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, ch. 20 (3e éd. : §22.4)

Problème 34 (La propriété de coupe, et pourquoi deux algorithmes gloutons fonctionnent tous deux — Licence). **DCS-34. Problème.** *Soit G un graphe fini connexe dont tous les poids d'arêtes sont distincts. Une coupe est une partition des sommets en deux parts non vides, et une arête traverse la coupe si ses extrémités sont dans des parts différentes.*

- (a) Démontrez la **propriété de coupe** : pour toute coupe, l'arête la moins chère qui la traverse appartient à tout arbre couvrant de poids minimal.
- (b) Démontrez la **propriété de cycle** : l'arête la plus chère de tout cycle n'appartient à aucun arbre couvrant de poids minimal.
- (c) Déduisez-en que l'arbre couvrant de poids minimal est unique, et tirez de (a) la correction à la fois de l'algorithme de Kruskal (ajouter l'arête la moins chère ne créant pas de cycle) et de l'algorithme de Prim (faire croître un seul arbre par son arête sortante la moins chère).
- (d) Montrez que l'unicité exige réellement des poids distincts, et précisez quelles parties de (a)–(c) survivent lorsque les poids peuvent être égaux.

Otakar Borůvka a posé et résolu le problème en 1926 pour une raison tout à fait pratique : la compagnie d'électricité morave voulait le réseau de lignes le moins cher reliant un ensemble de villes. Son algorithme a été redécouvert à répétition — par Jarník en 1930, par Kruskal en 1956, par Prim en 1957 et par Dijkstra en 1959 — histoire que Graham et Hell ont reconstituée en 1985, et avertissement sur la facilité avec laquelle de bonnes idées se perdent entre les langues et les décennies. La propriété de coupe est l'exemple le plus net, sur cette page, d'un argument *d'échange glouton* (comparez DCS-08) : pour voir qu'un choix glouton est sûr, on prend une solution optimale quelconque, on y échange l'arête gloutonne, et on montre que rien n'a empiré.

Références :

- Contexte : Arbre couvrant de poids minimal — Wikipédia (https://fr.wikipedia.org/wiki/Arbre_couvrant_de_poids_minimal)
- Contexte : Algorithme de Kruskal — Wikipédia (https://fr.wikipedia.org/wiki/Algorithme_de_Kruskal)
- Article : Graham & Hell, « On the history of the minimum spanning tree problem », *Annals of the History of Computing* 7 (1985)
- Lecture : Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, ch. 21 (3e éd. : ch. 23)

Problème 35 (Flot-max coupe-min, et pourquoi la réponse est entière — Licence). **DCS-43.**

Problème. Dans un graphe orienté muni d'une source s , d'un puits t et d'une capacité positive sur chaque arc, un flot respecte toutes les capacités et conserve la matière en tout sommet autre que s et t ; une coupe s - t est une partition (S, T) des sommets avec $s \in S$ et $t \in T$, et sa capacité est la capacité totale des arcs allant de S vers T . La méthode de Ford–Fulkerson pousse à répétition autant de flot qu'elle le peut le long d'un chemin s - t du graphe résiduel.

- (a) Démontrez que la valeur de tout flot est au plus la capacité de toute coupe, puis démontrez le théorème flot-max coupe-min sous sa forme fine : pour un flot f , les trois énoncés « f est de valeur maximale », « le graphe résiduel ne contient aucun chemin s - t » et « une coupe a pour capacité la valeur de f » sont équivalents.
- (b) Démontrez le théorème d'intégralité : si toutes les capacités sont entières, alors un flot maximal est entier, et la méthode ci-dessus en trouve un.
- (c) Déduisez de (a) et (b) le théorème des mariages de Hall et le théorème de Knig, en construisant un réseau adéquat à partir d'un graphe biparti.
- (d) Montrez qu'avec des capacités irrationnelles la méthode peut tourner indéfiniment sans converger vers le maximum, et démontrez qu'en augmentant toujours le long d'un plus court chemin résiduel — la règle d'Edmonds–Karp — le nombre d'augmentations est borné par $O(|V||E|)$, quelles que soient les capacités.

Ford et Fulkerson ont formulé et résolu le problème à la RAND en 1955–56, et la motivation n'avait rien d'académique : l'armée de l'air voulait savoir quelles liaisons ferroviaires du réseau soviétique et est-européen détruire pour étrangler le flux de ravitaillement au moindre coût. La coupe minimale était la liste des cibles, et l'étude Harris–Ross qui l'accompagnait, avec son goulot d'étranglement tracé sur une carte, est restée classifiée jusqu'en 1999. Elias, Feinstein et Shannon ont démontré le même théorème indépendamment cette année-là, par une voie relevant de la théorie de l'information. Le résultat est le visage combinatoire de la dualité en programmation linéaire, la question (c) le montre avalant tout entiers deux des théorèmes min–max classiques, et aujourd'hui toute routine de segmentation d'image, de couplage biparti ou de sélection de projets n'est que ce théorème en costume.

Références :

- Contexte : Théorème flot-max/coupe-min — Wikipédia (https://fr.wikipedia.org/wiki/Th%C3%A9or%C3%A8me_flot-max%2Fcoupe-min)
- Contexte : Algorithme de Ford–Fulkerson — Wikipédia (https://fr.wikipedia.org/wiki/Algorithme_de_Ford-Fulkerson)
- Article : Ford & Fulkerson, « Maximal flow through a network », Canadian Journal of Mathematics 8 (1956)
- Lecture : Kleinberg & Tardos, *Algorithm Design*, ch. 7

Problème 36 (Knuth–Morris–Pratt : la fonction d’échec et son automate — Licence). **DCS-44.**

Problème. Soit $p = p_1 p_2 \cdots p_m$ un motif. Un bord d’une chaîne est un préfixe propre qui en est aussi un suffixe ; soit $\pi(i)$ la longueur du plus long bord de $p_1 \cdots p_i$.

- Démontrez que les bords de $p_1 \cdots p_i$ sont exactement les chaînes de longueurs $\pi(i), \pi(\pi(i)), \pi(\pi(\pi(i))), \dots$ jusqu’à 0 — un seul tableau les enregistre donc tous.
- Concevez un algorithme calculant le tableau π tout entier en temps $O(m)$, démontrez sa correction, et démontrez la borne de temps par un argument amorti (trouvez la quantité qui croît d’au plus un par étape et ne peut jamais descendre sous zéro).
- Démontrez que le parcours d’un texte de longueur n qui en résulte ne revient jamais en arrière dans le texte et effectue moins de $2n$ comparaisons de caractères, de sorte que la recherche coûte $O(n + m)$ au total.
- Convertissez π en un véritable automate fini déterministe à $m + 1$ états, démontrez qu’il accepte exactement les chaînes se terminant par une occurrence de p , et comparez l’automate et la version à fonction d’échec en temps et en espace lorsque l’alphabet grandit.

L’algorithme a une généalogie singulièrement étrange. James Morris l’a implémenté en 1970 dans un éditeur de texte où — dit-on — un collègue l’a plus tard remplacé par la méthode naïve parce que personne n’y comprenait rien ; Donald Knuth est arrivé à la même procédure par l’autre bout, mécaniquement, à partir du théorème de Cook selon lequel les automates à pile déterministes bidirectionnels se simulent en temps linéaire — l’une des rares occasions où un résultat abstrait de complexité a livré un algorithme pratique. Vaughan Pratt les a rejoints et les trois ont publié en 1977. La question (b) est le plus petit spécimen net de la méthode du potentiel sur cette page après DCS-32, et la question (d) montre enfin l’automate fini de DCS-15 au travail rémunéré.

Références :

- Contexte : Algorithme de Knuth–Morris–Pratt — Wikipédia (https://fr.wikipedia.org/wiki/Algorithme_de_Knuth-Morris-Pratt)
- Article : Knuth, Morris & Pratt, « Fast pattern matching in strings », SIAM Journal on Computing 6 (1977)
- Lecture : Gusfield, *Algorithms on Strings, Trees, and Sequences*, ch. 2
- Lecture : Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, chapitre sur la recherche de motifs

Problème 37 (Les filtres de Bloom et le prix d’un faux positif — Licence, short). **DCS-45.**

Problème. Un filtre de Bloom représente un ensemble de n objets dans un tableau de m bits, tous nuls au départ, en mettant à 1 les bits situés aux positions $h_1(x), \dots, h_k(x)$ pour chaque x inséré, où les h_j sont des fonctions de hachage indépendantes à valeurs uniformes dans $\{1, \dots, m\}$; une requête répond « présent » lorsque ses k bits sont tous à 1. Démontrez qu’un faux négatif est impossible, qu’une requête portant sur un objet jamais inséré répond « présent » avec probabilité approximativement $(1 - e^{-kn/m})^k$, et que cette quantité est minimisée en prenant $k = (m/n) \ln 2$.

Burton Bloom a publié la structure en 1970 pour un dictionnaire de césures qui ne tenait pas en mémoire, échangeant un taux d'erreur petit et contrôlable contre un large gain en espace au facteur constant ; le même marché tourne aujourd'hui dans Cassandra, RocksDB et BigQuery pour éviter des lectures de disque inutiles, et dans les navigateurs qui confrontent des URL à des listes de blocage. Attention à la formule standard que l'on vous demande d'établir : elle traite les bits comme indépendants, ce qu'ils ne sont pas, et rendre l'analyse exacte a fait l'objet d'une note de 2008 dans *Information Processing Letters* montrant que l'expression familière sous-estime légèrement le taux réel. Un argument de théorie de l'information dit que toute structure de taux de faux positifs ε exige au moins $n \log_2(1/\varepsilon)$ bits ; le filtre de Bloom en dépense environ 1,44 fois plus, ce qui est précisément la raison d'être des filtres cuckoo et quotient.

Références :

- Contexte : Filtre de Bloom — Wikipédia (https://fr.wikipedia.org/wiki/Filtre_de_Bloom)
- Article : Bloom, « Space/time trade-offs in hash coding with allowable errors », Communications of the ACM 13 (1970)
- Lecture : Mitzenmacher & Upfal, *Probability and Computing*, ch. 5

Master

Problème 38 (Le problème de l'arrêt — Master). *DCS-17. Problème.* Fixez un modèle raisonnable de programmes capables de prendre des textes de programmes en entrée.

- (a) Démontrez que le problème de l'arrêt est indécidable : aucun programme H ne décide correctement, pour tout couple (programme P , entrée x), si P s'arrête sur x . Construisez vous-même le programme diagonal et suivez la contradiction dans les deux branches.
- (b) Démontrez que l'ensemble des arrêts est récursivement énumérable (un programme peut confirmer l'arrêt lorsqu'il survient), et concluez que son complémentaire ne l'est pas.
- (c) Montrez que l'indécidabilité est contagieuse : déduisez de (a) que « P s'arrête-t-il sur l'entrée vide ? » est également indécidable, par une réduction que vous construirez explicitement.

Turing l'a démontré en 1936, à 23 ans, pour trancher l'Entscheidungsproblem de Hilbert (Church l'a fait indépendamment par le λ -calcul) ; l'argument diagonal est celui de Cantor, transplanté de la théorie des ensembles vers les machines. C'est le théorème d'impossibilité fondateur de l'informatique — la ligne exacte où les mathématiques démontrent qu'aucune ingéniosité ne suffira jamais — et la question (c) est la première réduction du lecteur, l'outil qui fait tourner tout le reste de cette section.

Références :

- Contexte : Problème de l'arrêt — Wikipédia (https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_l%27arr%C3%AAt)
- Article : Turing, « On Computable Numbers, with an Application to the Entscheidungsproblem », Proceedings of the London Mathematical Society (1936)
- Lecture : Sipser, *Introduction to the Theory of Computation*, ch. 4
- Cours : MIT OCW 18.404J Theory of Computation (Sipser) (<https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/>)

Problème 39 (Réductions et théorème de Rice — Master). *DCS-18. Problème.* Appelons sémantique une propriété des programmes qui ne dépend que de la fonction partielle calculée par le programme, et non triviale une propriété que possède au moins une fonction calculable et que n'a pas au moins une autre.

- (a) Démontrez le théorème de Rice : toute propriété sémantique non triviale des programmes est indécidable, en construisant une réduction depuis le problème de l'arrêt — étant donné (P, x) , fabriquez un programme dont la fonction calculée possède la propriété si et seulement si P s'arrête sur x , en traitant avec soin le cas où la fonction nulle part définie possède elle-même la propriété.
- (b) Déduisez-en immédiatement que « affiche hello world », « calcule une fonction totale », « est un virus » (défini sémantiquement) sont toutes indécidables.
- (c) Exhibez deux propriétés décidables non sémantiques des textes de programmes et dites pourquoi le théorème de Rice ne s'y applique pas.

Henry Gordon Rice l'a démontré dans sa thèse de 1951 (publiée en 1953), et c'est l'indécidabilité industrialisée : non pas une question impossible mais toutes à la fois. Il trace la frontière exacte qui fait de la vérification de programmes un art plutôt qu'un algorithme — toute question intéressante sur ce que fait un code est formellement indécidable, ce qui explique pourquoi les vérificateurs réels approximent, restreignent, ou demandent l'aide du programmeur. La question (c) marque l'issue de secours : la syntaxe est décidable, seul le sens ne l'est pas.

Références :

- Contexte : Théorème de Rice — Wikipédia (https://fr.wikipedia.org/wiki/Th%C3%A9or%C3%A8me_de_Rice)
- Article : Rice, « Classes of recursively enumerable sets and their decision problems », Transactions of the AMS 74 (1953)
- Lecture : Sipser, *Introduction to the Theory of Computation*, ch. 5

Problème 40 (Cook–Levin : comprendre la réduction maîtresse — Master). **DCS-19. Problème.** Le théorème de Cook–Levin énonce que SAT — la satisfiabilité des formules booléennes — est NP-complet : tout problème vérifiable en temps polynomial s'y réduit. Ceci est un problème de compréhension : travaillez la démonstration par tableau (Sipser §7.4 ou Arora–Barak ch. 2), puis, à livre fermé, (a) expliquez pourquoi un calcul acceptant d'un vérificateur polynomial peut s'encoder en une table de cellules $\text{poly}(n) \times \text{poly}(n)$; (b) démontrez le lemme de localité — la correction de la table entière est imposée par des contraintes de taille constante sur des fenêtres 2×3 chevauchantes — et expliquez pourquoi c'est la taille constante qui rend la formule polynomiale ; (c) désignez l'endroit exact où la démonstration utilise que la machine est déterministe une fois son certificat donné ; (d) terminez : convertissez les contraintes de fenêtre en une formule sous forme normale conjonctive et vérifiez que la réduction s'exécute en temps polynomial.

Stephen Cook l'a démontré en 1971 à Berkeley (peu avant qu'on ne lui refuse la titularisation, événement que ses hôtes ont publiquement regretté depuis) ; Leonid Levin l'a démontré indépendamment en URSS en 1973, dans un article de deux pages que personne en Occident n'a lu pendant des années. La liste de 21 problèmes complets dressée par Karp en 1972 a ensuite montré que le théorème était un passe-partout et non une curiosité. Tout DCS-20, et le problème P contre NP lui-même, reposent sur cette construction — elle mérite une reconstruction honnête à livre fermé dans la vie de tout mathématicien.

Références :

- Contexte : Théorème de Cook–Levin — Wikipédia (https://fr.wikipedia.org/wiki/Th%C3%A9or%C3%A8me_de_Cook)
- Article : Cook, « The Complexity of Theorem-Proving Procedures », STOC 1971
- Lecture : Sipser, *Introduction to the Theory of Computation*, ch. 7
- Lecture : Arora & Barak, *Computational Complexity : A Modern Approach*, ch. 2

Problème 41 (Deux réductions de votre cru : 3SAT vers CLIQUE et VERTEX COVER — Master). **DCS-20. Problème.** En admettant la NP-complétude de 3SAT (DCS-19), démontrez la NP-complétude de :

- (a) *CLIQUE* — étant donné un graphe et k , existe-t-il un ensemble de k sommets deux à deux adjacents ? Construisez à partir d'une formule 3-CNF à m clauses un graphe tel qu'une clique de taille m existe si et seulement si la formule est satisfiable, et démontrez les deux sens.
- (b) *VERTEX COVER* — étant donné un graphe et k , existe-t-il un ensemble de k sommets touchant toute arête ? Démontrez d'abord l'identité de complémentarité : S est une clique de G si et seulement si $V \setminus S$ est une couverture par sommets du graphe complémentaire ; assemblez ensuite la réduction et démontrez-la.
- (c) Dites exactement ce qu'il faut vérifier, outre les réductions, avant de pouvoir prononcer le mot « NP-complet », et vérifiez-le.

Les deux réductions viennent de l'article de Karp de 1972, « Reducibility Among Combinatorial Problems », dont les 21 problèmes ont transformé le théorème de Cook en carte d'un monde intraitable entier ; le catalogue de Garey et Johnson en 1979 a porté la liste à plusieurs centaines, et elle se compte aujourd'hui par milliers. Faire soi-même deux réductions — inventer le gadget, démontrer les deux sens, sentir où la validité manque de fuir — est le seul moyen de faire passer cette technique des yeux aux mains.

Références :

- Contexte : Problème de la clique — Wikipédia (https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_la_clique)
- Contexte : Couverture par sommets — Wikipédia (https://fr.wikipedia.org/wiki/Couverture_par_sommets)
- Article : Karp, « Reducibility Among Combinatorial Problems » (1972)
- Lecture : Garey & Johnson, *Computers and Intractability : A Guide to the Theory of NP-Completeness*

Problème 42 (Approcher la couverture par sommets, et le mur du facteur 2 — Master). **DCS-21.**

Problème. (a) Démontrez que l'algorithme suivant produit une couverture par sommets valant au plus deux fois le minimum : prendre à répétition une arête non couverte, ajouter ses deux extrémités à la couverture, puis effacer toutes les arêtes qu'elles touchent. (La clé : les arêtes choisies forment un couplage ; toute couverture doit payer un sommet par arête couplée.) (b) Montrez que l'analyse est serrée — donnez une famille de graphes sur lesquels l'algorithme rend exactement le double de l'optimum. (c) Montrez que la rivale naturelle, « prendre à répétition le sommet de plus grand degré », est pire : son rapport croît comme $\ln n$. (d) Expliquez le mur : Dinur et Safra ont démontré (2005) qu'approcher en deçà de 1,36 est NP-difficile, Khot et Regev ont démontré (2008) que $2 - \varepsilon$ est difficile sous la conjecture des jeux uniques, et la démonstration en 2018 du théorème des jeux 2-vers-2 (Khot, Minzer, Safra et d'autres) a rendu inconditionnelle la difficulté à $\sqrt{2} - \varepsilon$. Dites précisément ce qui reste ouvert entre $\sqrt{2}$ et 2.

L'algorithme de couplage de la question (a) relève du folklore, attribué à Gavril et Yannakakis (années 1970), et son analyse en deux lignes contre un optimum invisible est le geste fondateur des algorithmes d'approximation : comparer à une borne inférieure, jamais à OPT lui-même. Que personne n'ait battu 2 — par aucun algorithme polynomial, en cinquante ans — tandis que les résultats de difficulté grimpent lentement vers cette valeur, constitue l'un des écarts ouverts les plus nets du domaine, et la raison pour laquelle la couverture par sommets est la première pièce exposée aussi bien dans les cours d'approximation que dans ceux de difficulté d'approximation.

Références :

- Contexte : Couverture par sommets — Wikipédia (https://fr.wikipedia.org/wiki/Couverture_par_sommets)
- Contexte : Conjecture des jeux uniques — Wikipédia (https://fr.wikipedia.org/wiki/Conjecture_des_jeux_uniques)
- Article : Khot & Regev, « Vertex cover might be hard to approximate to within $2-\varepsilon$ », Journal of Computer and System Sciences 74 (2008)
- Lecture : Vazirani, *Approximation Algorithms*, ch. 1

Problème 43 (Complexité de la communication : le coût de l'égalité — Master). **DCS-22. Problème.** Alice détient $x \in \{0,1\}^n$, Bob détient $y \in \{0,1\}^n$; ils échangent des bits selon un protocole déterministe jusqu'à ce que tous deux sachent si $x = y$.

- Démontrez qu'un tel protocole partitionne la matrice des entrées $2^n \times 2^n$ en rectangles combinatoires monochromes, un par transcription de communication.
- Démontrez la borne inférieure par ensemble trompeur : les 2^n entrées diagonales (x, x) trompent deux à deux tout rectangle, si bien que n bits sont nécessaires (en fait $n + 1$ avec la réponse).
- Autorisez maintenant l'aléa partagé et une erreur $1/3$: concevez un protocole n'échangeant que $O(1)$ bits, fondé sur des produits scalaires aléatoires, démontrez sa borne d'erreur et amplifiez-la.
- Concluez à l'écart de fond : l'égalité coûte n en déterministe et $O(1)$ en aléatoire — démontrez qu'aucun protocole déterministe ne peut même atteindre $n/2$.

Andrew Yao a fondé la complexité de la communication en 1979, en réduisant tout calcul réparti à une seule question : combien de bits doivent traverser le fil ? La géométrie des rectangles de la question (a) est le microscope du domaine, et l'égalité en est l'atome d'hydrogène — la première séparation entre calcul déterministe et calcul aléatoire que l'on puisse démontrer entièrement en une après-midi. Le modèle fournit aujourd'hui des bornes inférieures pour les algorithmes de flux, les structures de données, les tracés VLSI et la complexité des circuits ; le livre de Kushilevitz et Nisan en est la porte d'entrée standard.

Références :

- Contexte : Complexité de la communication — Wikipédia (https://fr.wikipedia.org/wiki/Complexité_de_la_communication)
- Article : Yao, « Some Complexity Questions Related to Distributive Computing », STOC 1979
- Lecture : Kushilevitz & Nisan, *Communication Complexity*, ch. 1–3
- Lecture : Arora & Barak, *Computational Complexity*, chapitre sur la complexité de la communication

Problème 44 (Presque toutes les chaînes sont incompressibles — Master). **DCS-23. Problème.** Fixez une machine universelle U ; la complexité de Kolmogorov $K(x)$ d'une chaîne x est la longueur du plus court programme qui fait imprimer x par U .

- Démontrez que $K(x) \leq |x| + c$ pour une constante c indépendante de x .
- Démontrez par dénombrement que pour tout n une chaîne de longueur n vérifie $K(x) \geq n$, et qu'au plus une fraction 2^{-k} des chaînes de longueur n vérifie $K(x) < n - k$: presque toutes les chaînes sont incompressibles.
- Démontrez que K n'est pas calculable, par un argument à la Berry : un programme capable de calculer K pourrait imprimer « la première chaîne de complexité au moins m » en bien moins de m bits.

- (d) Utilisez (b) pour donner en un paragraphe la démonstration qu'aucun compresseur sans perte ne réduit tous les fichiers, et expliquez pourquoi (c) n'empêche pas gzip d'exister.

La définition est due indépendamment à Solomonoff (1960), à Kolmogorov (1965) et à Chaitin ; c'est l'indépendance vis-à-vis de la machine, à une constante près, qui donne un sens à « la » complexité d'une chaîne. L'argument de dénombrement de la question (b) — trivial une fois vu, applicable sans fin — fait tourner la *méthode d'incompressibilité*, qui fournit de nettes démonstrations de bornes inférieures en combinatoire et en analyse d'algorithmes ; le livre de Li et Vitányi en rassemble des dizaines. La question (c) est le paradoxe de Berry promu du salon au théorème.

Références :

- Contexte : Complexité de Kolmogorov — Wikipédia (https://fr.wikipedia.org/wiki/Complexit%C3%A9_de_Kolmogorov)
- Article : Kolmogorov, « Three approaches to the quantitative definition of information », *Problems of Information Transmission* 1 (1965)
- Lecture : Li & Vitányi, *An Introduction to Kolmogorov Complexity and Its Applications*, ch. 1–2

Problème 45 (Myhill–Nerode — Master, short). **DCS-35. Problème.** Pour $L \subseteq \Sigma^*$, appelons équivalentes deux chaînes x, y lorsque $xz \in L \iff yz \in L$ pour tout $z \in \Sigma^*$. Démontrez que L est régulier si et seulement si cette équivalence a un nombre fini de classes, et que dans ce cas l'automate déterministe minimal de L a exactement un état par classe — il est donc unique à renommage près.

John Myhill (1957) et Anil Nerode (1958) ont fait passer « régulier » du statut de propriété des machines à celui de propriété du langage lui-même : les états que l'on ne peut éviter sont les futurs distincts que le langage peut avoir. C'est un instrument plus fin que le lemme de l'étoile de DCS-15, lequel ne donne qu'une condition nécessaire, et c'est le fondement théorique de la minimisation des automates déterministes — l'algorithme de Hopcroft de 1971 fusionne les états équivalents en temps $O(n \log n)$ et reste la référence.

Références :

- Contexte : Théorème de Myhill–Nerode — Wikipédia (https://fr.wikipedia.org/wiki/Th%C3%A9or%C3%A8me_de_Myhill-Nerode)
- Lecture : Sipser, *Introduction to the Theory of Computation*, ch. 1
- Lecture : Hopcroft, Motwani & Ullman, *Introduction to Automata Theory, Languages, and Computation*, ch. 4

Problème 46 (Trouver n'est pas plus dur que décider — Master, short). **DCS-36. Problème.** Supposez que vous puissiez demander à un oracle, au coût unitaire, si une formule booléenne donnée est satisfiable. Montrez que n appels à l'oracle et un travail polynomial supplémentaire suffisent à produire une affectation satisfaisante effective de toute formule satisfiable à n variables, et concluez que si $P = NP$ alors les affectations satisfaisantes peuvent être construites en temps polynomial, et non seulement détectées.

Cet argument d'auto-réductibilité — fixer une variable, demander si la formule simplifiée reste satisfiable, puis s'engager — est la raison pour laquelle la théorie de la complexité peut passer sa vie sur des questions par oui ou non sans rien perdre. C'est aussi la raison pour laquelle « $P = NP$ » serait catastrophique et non simplement intéressant : tout énoncé mathématique ayant une démonstration courte verrait cette démonstration trouvée, et pas seulement certifiée. Notez tout ce que l'argument utilise de SAT ; pour un problème quelconque de NP, on ne sait pas si la recherche se réduit à la décision.

Références :

- Contexte : Problème SAT — Wikipédia (https://fr.wikipedia.org/wiki/Probl%C3%A8me_SAT)
- Contexte : Problème fonctionnel (recherche contre décision) — Wikipédia (en anglais) (https://en.wikipedia.org/wiki/Function_problem)
- Lecture : Arora & Barak, *Computational Complexity : A Modern Approach*, ch. 2

Problème 47 (Union-find, et un algorithme qui n'est pas tout à fait linéaire — Master). **DCS-37. Problème.** Maintenez une partition de n éléments sous les opérations *find* (nommer la classe d'un élément) et *union* (fusionner deux classes), chaque classe étant représentée par un arbre enraciné. L'union par rang accroche la racine de plus petit rang sous la racine de plus grand rang, en tranchant les égalités par incrémentation du rang du survivant ; la compression de chemin fait pointer directement vers la racine tout nud rencontré par un *find*.

- Avec l'union par rang seule, démontrez qu'un arbre dont la racine a le rang r contient au moins 2^r nuds, et déduisez-en que chaque opération coûte $O(\log n)$.
- Démontrez qu'à tout instant au plus $n/2^r$ nuds ont le rang r .
- Avec les deux heuristiques, démontrez la borne de Hopcroft–Ullman : toute suite de $m \geq n$ opérations coûte $O(m \log^* n)$ au total, où $\log^* n$ est le nombre de fois qu'il faut appliquer $\log_2$ pour amener n sous 1. (Groupez les rangs en blocs et imputez chaque changement de pointeur soit à l'opération, soit à son nud.)
- Énoncez le théorème plus fin de Tarjan et le sens dans lequel il est optimal, et expliquez pourquoi aucune subtilité d'analyse ne peut ramener (c) à $O(m)$.

Galler et Fischer ont décrit la structure en 1964 ; Hopcroft et Ullman ont démontré la borne en $\log^*$ en 1973 ; et en 1975 Robert Tarjan a montré que le vrai coût amorti est $\Theta(\alpha(n))$ par opération, où α est la réciproque de la fonction d'Ackermann — une fonction si lentement croissante que $\alpha(n) \leq 4$ pour tout n qui sera jamais stocké dans un ordinateur, et pourtant non bornée, comme on le démontre. C'était la première fois qu'un algorithme naturel et pratique se révélait avoir une complexité non linéaire pour une raison que nul n'aurait pu deviner, et Fredman et Saks ont démontré en 1989 que la borne n'est pas un artefact de l'analyse : $\Omega(\alpha(n))$ sondages amortis sont nécessaires dans le modèle cell-probe. Toute implémentation de Kruskal (DCS-34) porte cette structure en elle.

Références :

- Contexte : Union-find (structure d'ensembles disjoints) — Wikipédia (<https://fr.wikipedia.org/wiki/Union-find>)
- Contexte : Fonction d'Ackermann — Wikipédia (https://fr.wikipedia.org/wiki/Fonction_d%27Ackermann)
- Article : Tarjan, « Efficiency of a good but not linear set union algorithm », *Journal of the ACM* 22 (1975)
- Lecture : Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, ch. 19 (3e éd. : ch. 21)

Problème 48 (Couverture par ensembles : relâcher, arrondir, et rencontrer l'écart — Master). **DCS-46. Problème.** Dans SET COVER on nous donne un univers U de n éléments et des ensembles $S_1, \dots, S_m \subseteq U$ de coûts $c_j \geq 0$, et l'on doit choisir des ensembles de coût total minimal dont la réunion vaut U . Le programme en nombres entiers naturel minimise $\sum_j c_j x_j$ sous les contraintes $\sum_{j: e \in S_j} x_j \geq 1$ pour tout $e \in U$, avec $x_j \in \{0, 1\}$; sa relaxation linéaire remplace cela par $0 \leq x_j \leq 1$.

- Démontrez que l'algorithme glouton — prendre à répétition l'ensemble qui minimise le coût par élément nouvellement couvert — rend une couverture de coût au plus $H_n = 1 + \frac{1}{2} + \dots + \frac{1}{n}$ fois l'optimum.

- (b) *Arrondi aléatoire* : résolvez la relaxation pour obtenir x^* , puis incluez indépendamment chaque S_j avec probabilité x_j^* , et répétez tout le tirage $\lceil 2 \ln n \rceil$ fois en prenant la réunion. Démontrez que le résultat couvre U avec probabilité au moins $1 - 1/n$ et a un coût espéré $O(\log n)$ fois la valeur de la relaxation.
- (c) Démontrez que la relaxation a un saut d'intégrité $\Omega(\log n)$ en exhibant une famille d'instances sur lesquelles l'optimum entier dépasse l'optimum fractionnaire de ce facteur — aucun arrondi plus habile de cette relaxation ne peut donc battre (b) en général.
- (d) Énoncez avec exactitude le versant difficulté, attributions et hypothèses comprises, puis expliquez soigneusement ce qu'un théorème de difficulté d'approximation ne dit pas sur telle instance particulière que l'on pourrait vous remettre.

La couverture par ensembles est l'un des vingt et un problèmes originels de Karp ; l'analyse logarithmique du glouton a été trouvée indépendamment par Johnson et Lovász au milieu des années 1970, puis étendue aux coûts par Chvátal. L'arrondi aléatoire est venu d'une direction inattendue — Raghavan et Thompson l'ont introduit en 1987 pour le routage global des fils dans les puces VLSI — et il est devenu l'une des deux ou trois techniques universelles de l'approximation. Réfermer l'écart du côté de la difficulté a pris vingt ans : Lund et Yannakakis (1994) puis Feige (1998) ont atteint $(1 - o(1)) \ln n$ sous une hypothèse quasi polynomiale, Raz et Safra ont obtenu une constante plus faible sous $P \neq NP$, et Dinur et Steurer ont finalement égalé exactement le glouton sous $P \neq NP$ en 2014. Bien peu de problèmes ont leur approximabilité épinglée avec cette précision ; comparez le mur obstiné du facteur 2 de DCS-21.

Références :

- Contexte : Problème de couverture par ensembles — Wikipédia (https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_couverture_par_ensembles)
- Contexte : Arrondi aléatoire — Wikipédia (en anglais) (https://en.wikipedia.org/wiki/Randomized_rounding)
- Lecture : Williamson & Shmoys, *The Design of Approximation Algorithms*, ch. 1
- Article : Dinur & Steurer, « Analytical approach to parallel repetition », STOC 2014

Problème 49 (Le théorème PCP sous ses deux formes — Master, short). **DCS-47. Problème.** *Le théorème PCP énonce que $NP = PCP(\log n, 1)$: tout langage de NP admet des démonstrations qu'un vérificateur aléatoire peut contrôler en ne lisant que $O(1)$ bits de la démonstration et en tirant $O(\log n)$ pièces, en acceptant toujours une démonstration correcte et en acceptant une démonstration fausse avec probabilité au plus $1/2$. Démontrez que cela équivaut à la forme dite « à écart » — pour une constante $\varepsilon > 0$, il est NP-difficile de distinguer les formules 3-CNF satisfiables de celles dont aucune affectation ne satisfait plus d'une fraction $1 - \varepsilon$ des clauses — et déduisez-en que MAX-3SAT n'admet aucun schéma d'approximation polynomial à moins que $P = NP$.*

Le théorème a été démontré en 1992 par Arora et Safra, conjointement avec Arora, Lund, Motwani, Sudan et Szegedy, couronnant la décennie des démonstrations interactives qui venait de produire $IP = PSPACE$, et il a reçu le prix Gödel 2001. Son effet fut de convertir l'inapproximabilité, jusque-là dispersée en résultats ad hoc, en machine : dès qu'un problème à écart est difficile, des réductions préservant l'approximation portent cette difficulté à des centaines de problèmes d'optimisation, dont la couverture par sommets de DCS-21 et la couverture par ensembles de DCS-46. Les résultats optimaux de Håstad en 1997 ont ensuite épinglé MAX-3SAT à exactement $7/8$ — affecter chaque variable au hasard est le mieux possible. La démonstration d'Irit Dinur en 2007 par amplification d'écart a remplacé l'argument original de cent pages par une récurrence combinatoire qui tient dans un cours magistral, et c'est la version à lire d'abord.

Références :

- Contexte : Théorème PCP — Wikipédia (https://fr.wikipedia.org/wiki/Th%C3%A9or%C3%A8me_PCP)
- Contexte : Difficulté d'approximation — Wikipédia (en anglais) (https://en.wikipedia.org/wiki/Hardness_of_approximation)
- Article : Dinur, « The PCP theorem by gap amplification », Journal of the ACM 54 (2007)
- Lecture : Arora & Barak, *Computational Complexity : A Modern Approach*, ch. 11

Problème 50 (Compter les éléments distincts en un seul passage — Master). ***DCS-48. Problème.** Un flux présente un à un des objets d'un univers $[u]$; soit D le nombre d'objets distincts qu'il contient. Vous ne pouvez stocker que bien moins que D et devez fournir une estimation $\hat{D}$ avec $|\hat{D} - D| \leq \varepsilon D$ avec probabilité au moins $2/3$. Fixez une fonction de hachage $h : [u] \rightarrow [0, 1]$, traitée comme donnant des valeurs uniformes indépendantes sur des arguments distincts.*

- Soit X la plus petite valeur de hachage rencontrée. Démontrez que $\mathbb{E}[X] = 1/(D + 1)$, et expliquez pourquoi $1/X - 1$ est un estimateur naturel mais désespérément bruité de D .
- Conservez plutôt les k plus petites valeurs de hachage et rendez $(k - 1)/X_{(k)}$, où $X_{(k)}$ est la k -ième plus petite et $D \geq k$. Démontrez que l'erreur relative est $O(1/\sqrt{k})$ avec probabilité constante, de sorte que $O(\varepsilon^{-2})$ valeurs stockées suffisent.
- Analysez plutôt l'estimateur de Flajolet–Martin : suivez le plus grand nombre R de zéros terminaux dans l'écriture binaire d'une valeur de hachage rencontrée, et démontrez que 2^R est à un facteur constant près de D avec probabilité constante. Expliquez ensuite pourquoi découper le flux en m seaux d'après les premiers bits de hachage puis combiner les maxima par seau à l'aide d'une moyenne harmonique est ce qui transforme cela en HyperLogLog, d'erreur relative environ $1,04/\sqrt{m}$.
- Démontrez une borne inférieure : tout algorithme déterministe décidant exactement si $D = n$ ou $D < n$ doit utiliser $\Omega(n)$ bits de mémoire, par une réduction depuis le problème de communication de DCS-22.

Flajolet et Martin ont inventé le comptage probabiliste au début des années 1980 pour répondre à la question d'un administrateur de bases de données — combien de valeurs distinctes cette colonne contient-elle? — en un seul passage et avec une poignée d'octets, et leur descendant HyperLogLog (Flajolet, Fusy, Gandouet et Meunier, 2007) estime des cardinalités de l'ordre du milliard à quelques pour cent près avec environ un kilo-octet et demi, ce qui explique sa présence dans Redis, Presto et toute grosse chaîne d'analyse de journaux. Le cadre théorique est venu plus tard : l'article de 1996 d'Alon, Matias et Szegedy sur les moments de fréquence a fondé le modèle des flux et a reçu le prix Gödel 2005, et Kane, Nelson et Woodruff ont donné en 2010 un algorithme d'éléments distincts d'espace optimal. La question (d) est l'honnête contrepoids — si tout cela est approché, c'est en vertu d'un théorème, non d'un manque d'efforts.

Références :

- Contexte : Problème du comptage d'éléments distincts — Wikipédia (en anglais) (https://en.wikipedia.org/wiki/Count-distinct_problem)
- Contexte : HyperLogLog — Wikipédia (<https://fr.wikipedia.org/wiki/HyperLogLog>)
- Article : Flajolet & Martin, « Probabilistic counting algorithms for data base applications », Journal of Computer and System Sciences 31 (1985)
- Article : Alon, Matias & Szegedy, « The space complexity of approximating the frequency moments », Journal of Computer and System Sciences 58 (1999)

Recherche

Problème 51 (P contre NP — Recherche). **DCS-24. Problème.** *A-t-on $P = NP$ — tout problème dont les solutions se vérifient en temps polynomial peut-il aussi se résoudre en temps polynomial ?*

Statut : ouvert ; c'est l'un des sept problèmes du prix du millénaire de l'institut Clay (1 000 000 \$), et l'attente générale est que $P \neq NP$. Les points d'entrée de cette page :

- (a) démontrez que si $P = NP$ alors la recherche se réduit à la décision — une affectation satisfaisante effective se trouve en temps polynomial en utilisant comme oracle un décideur de SAT ;
- (b) travaillez Baker–Gill–Solovay (1975) : construisez des oracles A et B avec $P^A = NP^A$ et $P^B \neq NP^B$, et expliquez précisément pourquoi cela tue toute technique de démonstration qui se relativise. Le traitement approfondi, avec l'histoire et les articles sur les barrières, se trouve sur la page Problèmes ouverts (open-problems.html).

Formulé indépendamment par Cook (1971) et Levin (1973), et étrangement anticipé dans une lettre de 1956 de Gödel à von Neumann demandant, en substance, si la démonstration de théorèmes pouvait se mécaniser en temps quadratique. Trois barrières nommées — la relativisation (1975), les démonstrations naturelles (Razborov–Rudich 1997) et l'algébrisation (Aaronson–Wigderson 2008) — expliquent pourquoi cinquante ans d'assauts n'ont pas abouti ; la question (b) est la première et la plus instructive d'entre elles, et c'est un travail honnête, voisin de la recherche, qu'un bon étudiant de master peut mener à bien.

Références :

- Contexte : Problème P = NP — Wikipédia (https://fr.wikipedia.org/wiki/Probl%C3%A8me_P_%E2%89%9F_NP)
- Lecture : Arora & Barak, *Computational Complexity*, ch. 2–3
- Article : Baker, Gill & Solovay, « Relativizations of the $P = ? NP$ Question », *SIAM Journal on Computing* 4 (1975)
- Lecture : Aaronson, « $P = ? NP$ », dans *Open Problems in Mathematics* (Springer, 2016)

Problème 52 (L'hypothèse du temps exponentiel — Recherche). **DCS-25. Problème.** *L'hypothèse du temps exponentiel (ETH) d'Impagliazzo et Paturi (2001) affirme que 3SAT exige un temps $2^{\delta n}$ pour un certain $\delta > 0$ (aucun algorithme sous-exponentiel) ; sa forme forte (SETH) affirme que la constante pour k SAT tend vers 2, celle de la force brute, quand $k \rightarrow \infty$. **Statut : les deux sont ouvertes** — strictement plus fortes que $P \neq NP$, ni démontrées ni réfutées, et devenues la monnaie de difficulté standard de la complexité fine. Points d'entrée :*

- (a) démontrez que ETH implique $P \neq NP$;
- (b) démontrez la réduction de Williams : SETH implique que le problème des vecteurs orthogonaux (étant donnés n vecteurs de $\{0,1\}^d$, $d = O(\log^2 n)$, décider si deux d'entre eux sont orthogonaux) n'admet pas d'algorithme en $O(n^{2-\varepsilon})$;
- (c) lisez comment Backurs–Indyk (2015) ont enchaîné de telles réductions jusqu'à la distance d'édition (DCS-10), rendant conditionnellement optimal un programme dynamique de 1974.

ETH convertit le « probablement difficile » en programme de recherche quantitatif : en supposant une borne inférieure précise au centre, des bornes inférieures de temps d'exécution serrées rayonnent vers l'extérieur jusqu'à des problèmes polynomiaux — distance d'édition, plus longue sous-suite commune, déformation temporelle dynamique — phénomène réellement nouveau des années 2010 que l'on appelle complexité fine. Le lemme de raréfaction qui rend ETH utilisable est lui-même un

théorème substantiel. Réfuter SETH, quant à elle, ferait la une : elle survit malgré des décennies d'ingénierie des solveurs SAT.

Références :

- Contexte : Hypothèse du temps exponentiel — Wikipédia (en anglais) (https://en.wikipedia.org/wiki/Exponential_time_hypothesis)
- Article : Impagliazzo & Paturi, « On the Complexity of k-SAT », Journal of Computer and System Sciences 62 (2001)
- Article : Backurs & Indyk, « Edit Distance Cannot Be Computed in Strongly Subquadratic Time (unless SETH is false) », STOC 2015
- Lecture : Arora & Barak, *Computational Complexity*, ch. 2 (prérequis)

Problème 53 (Le hasard aide-t-il ? BPP contre P — Recherche). **DCS-26. Problème.** *BPP est la classe des problèmes résolubles en temps polynomial par un algorithme à pile ou face avec une erreur d'au plus $1/3$ sur toute entrée. A-t-on $BPP = P$ — tout algorithme aléatoire efficace peut-il être dérandomisé ?* **Statut : ouvert, avec une forte réponse conditionnelle.** Points d'entrée :

- (a) démontrez l'amplification d'erreur — répéter puis prendre la majorité fait tomber l'erreur sous 2^{-n} (borne de Chernoff) ;
- (b) démontrez le théorème d'Adleman : $BPP \subseteq P/\text{poly}$ (le hasard peut être remplacé par un conseil) ;
- (c) comprenez le théorème d'Impagliazzo–Wigderson (1997) : s'il existe un problème en temps exponentiel exigeant des circuits de taille exponentielle, alors $BPP = P$ — la difficulté s'échange contre de la pseudo-aléa. La croyance en $BPP = P$ repose sur cet échange ; une démonstration inconditionnelle impliquerait des bornes inférieures de circuits que nul ne sait démontrer.

Dans les années 1970, le hasard ressemblait à une ressource de calcul au même titre que le temps — le test de primalité en était l'exemple vedette, avec Solovay–Strassen et Miller–Rabin battant tout ce qui était déterministe. Le pendule est reparti : les générateurs pseudo-aléatoires de Nisan–Wigderson ont montré que la difficulté se convertit en substituts de hasard, et en 2002 le test de primalité déterministe AKS a retiré le témoin vedette. Aujourd'hui la plupart des spécialistes pensent que le hasard n'achète au mieux que des accélérations polynomiales — mais le démontrer est bloqué, et cela se démontre, derrière le même mur des bornes inférieures de circuits que presque tout le reste de la complexité.

Références :

- Contexte : BPP (complexité) — Wikipédia ([https://fr.wikipedia.org/wiki/BPP_\(complexit%C3%A9\)](https://fr.wikipedia.org/wiki/BPP_(complexit%C3%A9)))
- Article : Impagliazzo & Wigderson, « $P = BPP$ if E Requires Exponential Circuits : Derandomizing the XOR Lemma », STOC 1997
- Lecture : Arora & Barak, *Computational Complexity*, ch. 7 et 20

Problème 54 (L'isomorphisme de graphes en temps quasi polynomial — Recherche). **DCS-27. Problème.** *Isomorphisme de graphes (GI) : étant donnés deux graphes à n sommets, décider s'il existe une bijection des sommets préservant l'adjacence. Statut : on ne le sait pas dans P ; on croit fermement qu'il n'est pas NP-complet (cela ferait s'effondrer la hiérarchie polynomiale, via l'appartenance de GI à coAM). Le jalon moderne : l'algorithme quasi polynomial de László Babai (2015–2017), de temps $\exp((\log n)^{O(1)})$ — en janvier 2017 Harald Helfgott a trouvé une faille dans l'analyse de la récursion lors de sa vérification pour le séminaire Bourbaki, et Babai a réparé l'algorithme en quelques jours ; le résultat corrigé tient, l'analyse publiée par Helfgott donnant $\exp(O((\log n)^3))$. Points d'entrée :*

- (a) démontrez que $GI \in NP$, et que compter les isomorphismes se réduit à décider ;
- (b) démontrez que l'isomorphisme d'arbres est en temps polynomial (l'algorithme AHU de forme canonique enracinée), camp de base de tout le sujet ;
- (c) lisez les sections de synthèse de Babai et repérez où la théorie des groupes (la méthode de Luks, les bornes sans classification) remplace la combinatoire. Ouvert : a-t-on $GI \in P$?

GI a passé quarante ans comme le plus célèbre candidat « intermédiaire » de la théorie de la complexité — trop structuré pour être NP-complet, trop insaisissable pour P — le $\exp(O(\sqrt{n} \log n))$ de Luks en 1983 restant le record jusqu'à ce que l'annonce de Babai électrise le domaine en novembre 2015. L'épisode faille-puis-réparation de 2017, mené entièrement en public, est aussi un modèle de la façon dont les mathématiques s'auto-corrigent à la frontière. Dans la pratique, cependant, des outils comme nauty résolvent instantanément d'énormes instances ; l'écart entre la facilité pratique et le statut théorique fait partie du charme du problème.

Références :

- Contexte : Problème de l'isomorphisme de graphes — Wikipédia (https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_l%27isomorphisme_de_graphes)
- Article : Babai, « Graph Isomorphism in Quasipolynomial Time » (2015), arXiv :1512.03547 (<https://arxiv.org/abs/1512.03547>)
- Article : Helfgott, « Isomorphismes de graphes en temps quasi-polynomial » (séminaire Bourbaki, 2017), arXiv :1701.04372 (<https://arxiv.org/abs/1701.04372>)
- Lecture : Arora & Barak, *Computational Complexity*, ch. 8 (démonstrations interactives, protocoles pour GI)

Problème 55 (L'exposant ω de la multiplication de matrices — Recherche). **DCS-28. Problème.** Soit ω la borne inférieure des exposants c tels que deux matrices $n \times n$ puissent être multipliées en $O(n^c)$ opérations arithmétiques. **Statut : ouvert ; la conjecture $\omega = 2$ n'est pas tranchée, et le record a bougé plusieurs fois depuis 2020.** On sait : $2 \leq \omega < 2,372$. Jalons : Strassen 1969 (2,81), Coppersmith–Winograd 1990 (2,376), puis, après 2020, une course de raffinements de la méthode laser — Alman–Williams 2020 (2,37286), Duan–Wu–Zhou 2022 (2,37188), Williams–Xu–Xu–Zhou, SODA 2024 (2,371552), avec d'autres raffinements en 2024–2026 (certains assistés par ordinateur) portant le record aux environs de 2,3712. Points d'entrée :

- (a) vérifiez l'identité à sept produits de Strassen pour des matrices par blocs 2×2 et démontrez qu'elle donne $O(n^{\log_2 7})$;
- (b) démontrez la borne inférieure facile $\omega \geq 2$;
- (c) lisez pourquoi la méthode laser elle-même ne peut atteindre 2 (barrière d'Ambainis–Filmus–Le Gall, 2015). Ouvert : la valeur de ω , et toute borne inférieure meilleure que la triviale.

L'article de Strassen de 1969 — intitulé « L'élimination de Gauss n'est pas optimale » — a lancé l'une des plus longues chasses au record des mathématiques, et l'exposant touche désormais à tout : algèbre linéaire rapide, fermeture transitive, analyse syntaxique, et entraînement des réseaux de neurones qui, retournement de 2022, ont eux-mêmes rejoint la chasse (AlphaTensor de DeepMind a trouvé de nouvelles décompositions tensorielles en petite dimension, et l'optimisation assistée par machine a contribué aux bornes les plus récentes). La plupart des spécialistes parient sur $\omega = 2$ assorti d'un lourd prix logarithmique ; personne n'a la moindre idée de la façon de démontrer une borne inférieure surquadratique. Vérifiez le record courant avant de le citer — le nombre de ce paragraphe a une date de péremption.

Références :

- Contexte : Complexité de la multiplication de matrices — Wikipédia (https://fr.wikipedia.org/wiki/Complexit%C3%A9_de_la_multiplication_de_matrices)

- Article : Vassilevska Williams, Xu, Xu & Zhou, « New Bounds for Matrix Multiplication : from Alpha to Omega », SODA 2024, arXiv :2307.07970 (<https://arxiv.org/abs/2307.07970>)
- Article : Strassen, « Gaussian elimination is not optimal », Numerische Mathematik 13 (1969)
- Lecture : Bürgisser, Clausen & Shokrollahi, *Algebraic Complexity Theory*

Problème 56 (3SUM et la barrière quadratique — Recherche, short). **DCS-38. Problème.** *Le problème 3SUM demande, étant donnés n entiers, si trois d'entre eux sont de somme nulle ; la conjecture 3SUM affirme qu'aucun algorithme ne s'exécute en temps $O(n^{2-\varepsilon})$ pour une constante $\varepsilon > 0$. Démontrez la borne supérieure $O(n^2)$, et démontrez qu'un algorithme véritablement sous-quadratique pour 3SUM en donnerait un pour décider si n points du plan comportent trois points alignés — par exemple au moyen de l'application $x \mapsto (x, x^3)$.*

Gajentaan et Overmars ont introduit la 3SUM-difficulté en 1995 comme moyen d'expliquer pourquoi des dizaines de problèmes naturels de géométrie algorithmique semblaient tous bloqués à n^2 ; la conjecture est aujourd'hui l'un des trois piliers de la complexité fine, aux côtés de SETH (voir DCS-25) et de la conjecture des plus courts chemins entre toutes les paires. Elle reste ouverte, et une comptabilité honnête s'impose ici : Grønlund et Pettie ont montré en 2014 que 3SUM *peut* se résoudre un peu plus vite que n^2 , à des facteurs polylogarithmiques près, ce qui réfute la forme naïve de la conjecture tout en laissant intacte la forme $n^{2-\varepsilon}$ — celle qu'utilisent toutes les réductions.

Références :

- Contexte : 3SUM — Wikipédia (en anglais) (<https://en.wikipedia.org/wiki/3SUM>)
- Article : Gajentaan & Overmars, « On a class of $O(n^2)$ problems in computational geometry », Computational Geometry 5 (1995)
- Article : Grønlund & Pettie, « Threesomes, Degenerates, and Love Triangles », FOCS 2014 / Journal of the ACM 65 (2018)

Problème 57 (La conjecture du log-rang — Recherche, short). **DCS-39. Problème.** *Pour $f : X \times Y \rightarrow \{0,1\}$, soit M_f la matrice $(f(x,y))_{x,y}$ sur les réels et soit $D(f)$ le nombre de bits que deux joueurs doivent échanger dans le pire cas pour calculer f . Démontrez la borne inférieure par le rang $D(f) \geq \log_2 \text{rank}(M_f)$, puis énoncez la conjecture du log-rang — à savoir que réciproquement $D(f) = O((\log \text{rank}(M_f))^c)$ pour une constante absolue c — accompagnée de la meilleure borne connue aujourd'hui.*

Lovász et Saks ont posé la conjecture en 1988, et elle est toujours ouverte : elle demande si le seul obstacle à une communication bon marché est d'ordre linéaire-algébrique, ce qui représenterait une quantité stupéfiante de structure à trouver dans une matrice booléenne arbitraire. La meilleure borne supérieure générale est le $D(f) = O(\sqrt{r} \log r)$ de Shachar Lovett avec $r = \text{rank}(M_f)$ — exponentiellement loin de la conjecture — et un exemple de Kushilevitz montre que la constante c ne peut être prise égale à 1, de sorte que le polynôme de l'énoncé n'est pas cosmétique. Comparez DCS-22, où la borne par le rang règle exactement le cas de l'égalité.

Références :

- Contexte : Conjecture du log-rang — Wikipédia (en anglais) (https://en.wikipedia.org/wiki/Log-rank_conjecture)
- Article : Lovett, « Communication is bounded by root of rank », Journal of the ACM 63 (2016)
- Lecture : Kushilevitz & Nisan, *Communication Complexity*, ch. 1–2

Problème 58 (Optimalité dynamique : les arbres splay sont-ils le meilleur arbre ? — Recherche). **DCS-40. Problème.** *Un arbre binaire de recherche sert une suite de m accès à des clés de*

$\{1, \dots, n\}$, en payant la profondeur de chaque nud accédé plus le nombre de rotations qu'il effectue. Le splaying remonte le nud accédé jusqu'à la racine par pas doubles (zig-zig et zig-zag) ; soit OPT le coût du meilleur algorithme hors ligne, qui voit d'avance toute la suite.

- (a) Démontrez le lemme d'accès : avec des poids $w_i > 0$, un poids total W et le potentiel $\Phi = \sum_v \log_2(\text{poids du sous-arbre en } v)$, le coût amorti du splaying du nud de clé i vaut au plus $3 \log_2(W/w_i) + 1$.
- (b) Déduisez-en la borne amortie $O(\log n)$ puis, par un choix des poids, l'optimalité statique : le coût total vaut $O(m + \sum_i m_i \log(m/m_i))$ lorsque la clé i est accédée m_i fois.
- (c) Montrez que sur une suite d'accès uniformément aléatoire tout algorithme d'arbre binaire de recherche paie $\Omega(\log n)$ amorti, de sorte que l'optimalité ne peut avoir de sens que comme compétitivité vis-à-vis de OPT sur des suites structurées.
- (d) Énoncez précisément la conjecture d'optimalité dynamique, et rapportez son statut avec exactitude.

Sleator et Tarjan ont inventé les arbres splay en 1985 — un arbre de recherche qui ne stocke aucune information d'équilibrage et se contente de hisser à la racine ce que vous venez de toucher — et ont conjecturé que cette règle fruste reste à un facteur constant près du meilleur arbre possible sur toute suite d'accès. La conjecture est ouverte. Elle a résisté quatre décennies : les arbres Tango de Demaine, Harmon, Iacono et Patrascu (2004) ont donné le premier algorithme $O(\log \log n)$ -compétitif, mais on ne savait rien du splaying lui-même au-delà de $O(\log n)$ jusqu'en juillet 2026, quand Chmél, Haeupler, Hladík, Koucký, Roeyskoe, Rozho, Sladký et Tarjan ont démontré que les arbres splay sont $\tilde{O}(\log \log n)$ -compétitifs. C'est dans l'écart entre $\log \log n$ et $O(1)$ que le problème vit encore.

Références :

- Contexte : Arbre splay — Wikipédia (https://fr.wikipedia.org/wiki/Arbre_splay)
- Article : Sleator & Tarjan, « Self-adjusting binary search trees », Journal of the ACM 32 (1985)
- Article : Demaine, Harmon, Iacono & Patrascu, « Dynamic optimality — almost », SIAM Journal on Computing 37 (2007)
- Article : Chmél, Haeupler, Hladík, Koucký, Roeyskoe, Rozho, Sladký & Tarjan, « Splay trees are almost dynamically optimal » (2026), arXiv :2607.18498 (<https://arxiv.org/abs/2607.18498>)

Problème 59 (La conjecture des k serveurs — Recherche). **DCS-49. Problème.** Dans le problème des k serveurs, k serveurs occupent des points d'un espace métrique ; des requêtes arrivent une à une en des points de l'espace, et chacune doit être servie en amenant un serveur dessus, à un coût égal à la distance parcourue. Un algorithme en ligne ne découvre chaque requête qu'à son arrivée. **Statut : ouvert.** La conjecture des k serveurs de Manasse, McGeoch et Sleator (1988) affirme qu'un algorithme déterministe est k -compétitif dans tout espace métrique.

- (a) Démontrez le cas de la pagination, où toutes les distances sont égales : LRU et FIFO sont tous deux k -compétitifs sur un cache de k pages. (Découpez la suite de requêtes en phases contenant chacune k pages distinctes, et faites les comptes par phase.)
- (b) Démontrez la borne inférieure correspondante pour la pagination : aucun algorithme déterministe en ligne n'a un rapport de compétitivité inférieur à k , au moyen d'un adversaire qui demande toujours une page que l'algorithme ne détient pas.
- (c) Étendez (b) au problème général : dans tout espace métrique d'au moins $k + 1$ points, aucun algorithme déterministe en ligne ne fait mieux que k .
- (d) Problème de compréhension : lisez comment Koutsoupias et Papadimitriou ont démontré en 1995 que l'algorithme de la fonction de travail est $(2k - 1)$ -compétitif, puis rapportez la moitié aléatoire de l'histoire en énonçant honnêtement le statut.

Statut, honnêtement énoncé : la conjecture déterministe est ouverte, et c'est le problème ouvert central des algorithmes en ligne. Sleator et Tarjan ont introduit l'analyse compétitive en 1985 pour lever une gêne persistante — LRU se comporte magnifiquement en pratique alors que l'analyse du pire cas déclare exécrable toute règle de pagination — et Manasse, McGeoch et Sleator ont généralisé la pagination à k serveurs dans un espace métrique trois ans plus tard, conjecturant au passage que k est toujours atteignable. Près de quarante ans après, la borne $(2k - 1)$ de l'algorithme de la fonction de travail n'a pas été améliorée, et sa démonstration reste parmi les arguments les plus ardues du domaine. Le versant aléatoire s'est achevé autrement et récemment : la conjecture aléatoire des k serveurs, selon laquelle un algorithme aléatoire $O(\log k)$ -compétitif existe dans tout espace métrique, a été *réfutée* par Bubeck, Coester et Rabani en 2022 — cas rare d'une conjecture algorithmique de longue date se révélant fausse.

Références :

- Contexte : Problème des k serveurs — Wikipédia (en anglais) (https://en.wikipedia.org/wiki/K-server_problem)
- Contexte : Algorithme de remplacement de pages — Wikipédia (en anglais) (https://en.wikipedia.org/wiki/Page_replacement_algorithm)
- Article : Sleator & Tarjan, « Amortized efficiency of list update and paging rules », Communications of the ACM 28 (1985)
- Article : Bubeck, Coester & Rabani, « The randomized k -server conjecture is false! » (2022), arXiv :2211.05753 (<https://arxiv.org/abs/2211.05753>)

Problème 60 (Jusqu'où peuvent aller les bornes inférieures cell-probe? — Recherche, short). ***DCS-50. Problème.** Dans le modèle cell-probe, la mémoire est découpée en cellules de w bits et seuls les accès mémoire sont facturés, tout calcul étant gratuit; une structure dynamique de sommes préfixes maintient $a_1, \dots, a_n$ sous les mises à jour $a_i \leftarrow a_i + \delta$ et les requêtes $\sum_{j \leq i} a_j$. Démontrez la borne supérieure $O(\log n)$ à l'aide d'un arbre équilibré de sommes partielles, puis énoncez la borne inférieure cell-probe correspondante $\Omega(\log n)$ de Ptracu et Demaine et rapportez honnêtement jusqu'où l'on est parvenu vers une borne inférieure polynomiale pour un problème dynamique explicite.*

Statut, honnêtement énoncé : les bornes polylogarithmiques constituent le record, et une borne polynomiale pour un quelconque problème dynamique explicite est grande ouverte. Andrew Yao a introduit ce modèle en 1981, dans un article demandant si les tables doivent être triées, précisément pour démontrer des bornes inférieures valables contre tout algorithme, si habile soit-il — en ne facturant que les accès mémoire on concède tout le reste et l'on peut malgré tout dire quelque chose, ce qui est plus que n'ont réussi les modèles de circuits ou de machines de Turing. La méthode du chronogramme de Fredman et Saks (1989) a donné les premières bornes $\Omega(\log n / \log \log n)$ pour les sommes partielles et pour le problème union-find de DCS-37; Ptracu et Demaine ont affiné les sommes partielles jusqu'à un $\Theta(\log n)$ serré. Depuis, le plafond n'a presque pas bougé : Larsen a démontré $\Omega((\log n / \log \log n)^2)$ pour le comptage pondéré sur intervalles en 2012 et, pour les problèmes dont la réponse tient en un bit, où $\Omega(\log n)$ faisait barrage, Larsen, Weinstein et Yu ont atteint $\Omega(\log^{1.5} n)$ en 2018. Que le modèle ne parvienne toujours pas à livrer une borne n^ϵ pour un problème explicite est, avec les bornes inférieures de circuits, l'une des gênes les plus vives de la théorie de la complexité.

Références :

- Contexte : Modèle cell-probe — Wikipédia (en anglais) (https://en.wikipedia.org/wiki/Cell-probe_model)
- Article : Fredman & Saks, « The cell probe complexity of dynamic data structures », STOC 1989
- Article : Ptracu & Demaine, « Logarithmic lower bounds in the cell-probe model », SIAM Journal on Computing 35 (2006)

Pour aller plus loin

Hors de la Caverne — des problèmes, pas de réponses.